

Algoritmo e Lógica de Programação

Prof. Me. Rodrigo Franklin



Gestão da Educação a Distância
Cidade Universitária – Bloco C
Avenida Alzira Barra Gazzola, 650,
Bairro Aeroporto. Varginha /MG
ead.unis.edu.br
0800 283 5665

Todos os direitos desta edição ficam reservados ao Unis – MG.

É proibida a duplicação ou reprodução deste volume (ou parte do mesmo), sob qualquer meio, sem autorização expressa da instituição.

Reitor

Prof. Me. Stefano Barra Gazzola

Vice-Reitor

Prof. Me. Jeferson Vinhas Ferreira

Gestão da Educação a Distância

Prof. Dr. Wanderson Gomes de Souza

Supervisão Técnica

Profa. Dra. Simone de Paula
Teodoro Moreira

Design Educacional

Rebeca Nogueira Lourenço Kaus

Design Instrucional e Diagramação

Amanda Alves Figueiredo

Revisão Ortográfica/Gramatical

Natáscia Destéfani

Kamilla Beatriz Campos

Autoria

Prof. Me.

Rodrigo Franklin Frogeri



Mestre em Administração com ênfase em Governança de Tecnologia da Informação. Especialista em Gestão de Tecnologia da Informação. Especialista em Docência no Ensino Superior. Especialista em Redes de Computadores. Bacharel em Ciência da Computação. Professor nos cursos de Bacharelado em Sistemas de Informação, Bacharelado em Ciência da Computação e Tecnólogo em Análise e Desenvolvimento de Sistemas. Professor nos cursos de Pós-Graduação de MBA Executivo em Gestão e Governança de Tecnologia da Informação, Tecnologia e Gerenciamento de Redes de Computadores e Cibersegurança e Perícia Forense Computacional, todos oferecidos pelo UNIS-MG.

Currículo Lattes: <http://lattes.cnpq.br/0403458828547468>



FROGERI, Rodrigo Franklin. Guia de Estudo – **Algoritmo e Lógica de Programação**. Varginha: GEaD-UNIS/MG, 2017.

129 p.

1. Algoritmo 2. Lógica 3. Desenvolvimento 4. Programação

Apresentação

“A primeira regra de qualquer tecnologia utilizada nos negócios é que a automação aplicada a uma operação eficiente aumentará a eficiência. A segunda é que a automação aplicada a uma operação ineficiente aumentará a ineficiência.”

Bill Gates

Prezado (a) Aluno (a),

Este é o seu Guia de Estudos da disciplina de Princípios de Desenvolvimento de Algoritmos. O Guia irá auxiliá-lo no desenvolvimento das competências necessárias para a construção lógica de programas de computador.

Cada disciplina do curso tem características e importância peculiares. Especificamente a disciplina de Princípios de Desenvolvimento de Algoritmos, que irá prepará-lo para os desafios que envolvem o projeto e desenvolvimento de programas. Com o auxílio deste guia, você vai aprender os passos básicos, a lógica e as técnicas para construção de algoritmos e programas de computador.

Esses desafios não são simples e para superá-los você precisará de uma boa iniciação de lógica e algoritmo. Seu primeiro desafio é encarar a produção de algoritmos de frente e superar as dificuldades naturais. Muitos ex-alunos consideram essa disciplina um “bicho de sete cabeças”, mas, como também sou ex-aluno da disciplina, posso afirmar que é apenas falta de empenho.

O desafio está lançado! Conte comigo!

Abraço, Prof. Rodrigo Franklin Frogeri.

Ementa

Princípios de lógica de programação. Metodologia de Desenvolvimento de Algoritmos. Tipos de Dados e Tipos abstratos de dados. Estruturas Sequenciais, Condicionais e de Repetição. Arquivos. Procedimentos e funções. Introdução à linguagem C. Estratégias de Depuração.

Orientações

Ver Plano de Estudos da disciplina, disponível no Ambiente Virtual.

Palavras-chave

Algoritmo. Programação. Computadores.



Sumário

EMENTA	5
ORIENTAÇÕES	5
PALAVRAS-CHAVE	5
UNIDADE I – INTRODUÇÃO A ALGORÍTMOS	9
1. INTRODUÇÃO	10
1.1 CONCEITOS BÁSICOS	10
1.2 INTRODUÇÃO A ALGORITMOS	13
1.3 ALGORITMO X PROGRAMA DE COMPUTADOR	18
1.4 CONSTRUINDO ALGORITMOS	20
1.5 TIPOS DE ALGORITMOS	23
1.6 EXEMPLOS DE ALGORITMOS	25
1.7 TESTE DE MESA	27
UNIDADE II – FUNDAMENTOS DE LÓGICA DE PROGRAMAÇÃO	28
2. INTRODUÇÃO	29
2.1 INTRODUÇÃO À LÓGICA DE PROGRAMAÇÃO	29
2.2 TIPOS DE DADOS	30
2.3 TIPOS DE CONSTANTES	35
2.4 DECLARAÇÃO DE VARIÁVEIS	37
2.4.1 COMENTÁRIOS EM ALGORITMOS	43
2.5 SOFTWARES DE APOIO	45
UNIDADE III – OPERADORES, EXPRESSÕES E COMANDOS BÁSICOS	46
3. INTRODUÇÃO	47
3.1 OPERADORES	47
3.2 EXPRESSÕES ARITMÉTICAS	52
3.3 FUNÇÕES ARITMÉTICAS	55
3.4 PRIORIDADES	56
3.5 COMANDO DE ATRIBUIÇÃO	58
3.6 COMANDOS DE ENTRADA E SAÍDA	64
3.6.1 COMANDO DE ENTRADA	64
3.6.2 COMANDO DE SAÍDA	67
UNIDADE IV – ESTRUTURAS BÁSICAS DE CONTROLE	70
4.1 SEQUÊNCIA	71

4.2 ESTRUTURA CONDICIONAL (ESTRUTURA DE SELEÇÃO)	72
4.2.1 ESTRUTURA CONDICIONAL SIMPLES	73
4.3 ESTRUTURA CONDICIONAL COMPOSTA	79
4.4 ESTRUTURAS DE REPETIÇÃO	81
4.4.1 COMANDO REPITA	84
4.4.2 A ESTRUTURA DE REPETIÇÃO ENQUANTO	93
4.4.3 ESTRUTURA DE REPETIÇÃO PARA	95
4.5 REGRAS PARA A CONSTRUÇÃO DE ALGORITMOS LEGÍVEIS	96
4.5.1 TIPOS ABSTRATOS DE DADOS	99
4.5.2 VETOR (MATRIZ UNIDIMENSIONAL)	99
4.5.3 MATRIZ (MATRIZ MULTIDIMENSIONAL)	102
UNIDADE V – INTRODUÇÃO À LINGUAGEM DE PROGRAMAÇÃO C	105
5.1 O BÁSICO DA LINGUAGEM C	106
5.2 A LINGUAGEM C É “CASE SENSITIVE”	107
5.3 CONFIGURANDO O AMBIENTE DE PROGRAMAÇÃO	107
5.4 TIPOS DE DADOS, EXPRESSÕES E OPERADORES EM C	110
5.5 INSTRUÇÃO DE ENTRADA SCANF	114
5.6 INSTRUÇÃO DE SAÍDA PRINTF	116
5.7 USO DE FUNÇÕES PRÉ-DEFINIDAS	119
REFRÊNCIA BIBLIOGRÁFICA	129

ÍCONES



APLICAÇÃO PROFISSIONAL. Indica uma aplicação prática de uso profissional ligada ao que está sendo estudado.



HIPERLINK. Indica um link, seja ele para outra página do livro ou endereço de internet.



ASSISTA. Indica o link de algum vídeo que está relacionado com o tema estudado.



IMPORTANTE. Aponta uma observação significativa. Pode ser encarado como um sinal de alerta que o orienta para prestar atenção à informação indicada.



CHECKLIST OU PROCEDIMENTO. Indica um conjunto de ações para fins de verificação de rotina ou um procedimento de uma tarefa.



PENSE. Indica que você deve refletir sobre o assunto abordado para responder a um questionamento.



CONCLUSÃO. Todas as conclusões, sejam de ideais, partes ou unidades do curso virão precedidas desse ícone.



PERFIL DO PROFISSIONAL. Indica as características e habilidades que o aluno deve adquirir e/ou desenvolver para ser um bom profissional da área.



DEBATE. Traz para o estudo do conteúdo, questões polêmicas e muitas vezes antagônicas, para que o aluno reflita.



PESQUISE. Indica a exigência de pesquisa a ser realizada na busca por mais informações.



DESAFIO. Lança um exercício ou aplicação de conteúdo mais complexo, que deve ser vencido pelo aluno.



REALIZE. Determina a existência de atividade teórica ou prática a ser realizada.



DICA. Orienta a perspectiva que determinado conteúdo deve ser entendido ou aplicado.



REALIZE MUSICAL. Determina a existência de atividade a ser realizada na área musical.



DOWNLOAD. Conteúdo que deve ser baixado da internet ou do AVA para estudo ou desenvolvimento da disciplina.



REVENDO. Indica a necessidade de rever conceitos estudados anteriormente.



ESCUTE. Arquivo de áudio que deve ser ouvido para dar continuidade ao estudo do conteúdo.



SAIBA MAIS. Apresenta informações adicionais sobre o tema abordado de forma a possibilitar a obtenção de novas informações ao que já foi referenciado.



EXEMPLO. Esse ícone será usado sempre que houver necessidade de exemplificar um caso, uma situação ou conceito que está sendo descrito ou estudado.



SUGESTÃO DE LEITURA. Indica textos de referência utilizados no curso e também faz sugestões para leitura complementar.



I

Unidade I - Introdução a Algoritmos



Objetivos da Unidade

- ✓ Apresentar os conceitos básicos acerca dos programas de computador.
- ✓ Diferenciar algoritmo de programa de computador.
- ✓ Apresentar os princípios da lógica de programação.



Plano de Estudos

- ✓ Ciclo 01
- ✓ Atividade Teste
- ✓ Título: Introdução a Lógica de Programação

I. Introdução

Nesta primeira unidade você aprenderá os conceitos básicos, definição de algoritmo, diferença entre algoritmo e programa de computador e, por fim, uma introdução à Lógica de Programação.

I.1 Conceitos Básicos

Em todas as disciplinas do curso vocês encontrarão alguns conceitos-chave que são base para os Sistemas de Informação. Os principais conceitos que estão relacionados diretamente ao desenvolvimento de programas de computador são:

Figura 01: Códigos de programação na tela de um computador



Istock.com

- **Programa de Computador:** sequência de instruções não ambígua e finita em uma linguagem de programação específica que, quando executada, resolve um determinado problema. Um software pode compreender diversos programas de computador.

- **Linguagem de máquina:** é a linguagem binária (composta de zeros e uns) utilizada pelos computadores, para representar dados, programas e informações.
- **Linguagem Assembler:** é uma linguagem particular para cada processador (computador), que codifica as instruções dos programas em linguagem de máquina. É considerada uma linguagem de baixo nível, pois os programas são criados com instruções compreendidas pelo processador. A programação em Assembler implica em manipulação direta dos endereços de memória e da alocação dos dados nos registradores do processador.
- **Linguagem de programação de 3ª geração:** também chamada de linguagem de alto nível, engloba todas as linguagens de programação que utilizam compiladores ou interpretadores (ver diferença a seguir). Possuem instruções mais poderosas que a linguagem Assembler, facilitando o trabalho do programador. Nestas linguagens, a manipulação da memória principal e registradores é abstraída do programador, ficando a cargo do compilador a geração de um programa executável.



Exemplos: Fortran, Cobol, Pascal, C, entre outras.

- **Linguagem de programação de 4ª geração:** são linguagens não procedurais, que utilizam o paradigma de orientação a objetos e a técnica de desenvolvimento chamada de RAD (*Rapid Application Development*), otimizando assim, o desenvolvimento de programas de computador. Linguagens desta geração, geralmente, utilizam IDE's (*Integrated Development Environment*), que são

ambientes de programação com diversas ferramentas para auxiliar o desenvolvimento RAD.



Alguns exemplos de IDE's são: Visual Studio, NetBeans, Eclipse, entre outros.

Exemplos de linguagens de 4ª geração: SQL, Java, C#. Python, entre outras.

- **Linguagem de programação de 5ª geração:** são linguagens orientadas à inteligência artificial (IA), como por exemplo: LISP (*LIS*t *PRO*cessing) e PROLOG (*PRO*gramming in *LOG*ic) (MATHIAS, 2015). Apesar de existirem linguagens originalmente voltadas para a inteligência artificial, linguagens de terceira e quarta geração podem ser utilizadas com algoritmos de IA, como por exemplo, as redes neurais.
- **Compilador:** é um aplicativo utilizado para traduzir os programas escritos pelo programador nas linguagens de alto nível (programa fonte) para a linguagem de máquina (programa executável), a fim de poder ser executado pelo computador. Os compiladores geram o chamado “programa objeto”, que por sua vez, passa pela etapa de linkedição, sendo criado assim o programa executável. Uma vez gerado o programa executável, este pode ser instalado nos hardwares que possuem o processador compatível com o programa “linkeditado”. Caso seja necessária uma adaptação para outro hardware, deverá ser gerado um novo executável.
- **Interpretador:** é um aplicativo que traduz os programas escritos pelo programador para a linguagem de máquina, porém não é gerado um programa executável como no caso de um programa compilado. Programas interpretados, geralmente, caracterizam-se por necessitar de uma espécie de máquina virtual, que será a responsável por ler o programa interpretado e executá-lo nesta máquina que, por sua vez, exibirá o programa na máquina

real. A literatura considera a execução dos programas interpretados mais lenta que a dos programas compilados, mas esta é uma afirmação que gera infinitas discussões e é alvo de grande polêmica no meio computacional, pois diversas variáveis e ambientes precisam ser avaliados para se chegar a esta conclusão, não sendo um consenso na comunidade acadêmica que programas compilados são mais rápidos para executar que programas interpretados. Por ser uma discussão que não se esgotou, vamos considerar que em condições de execução de um mesmo programa e com a utilização de variáveis estáticas em ambos, o programa compilado será mais rápido por não necessitar da leitura por parte de um terceiro componente.

I.2 Introdução a Algoritmos

Existem diversas definições para o termo Algoritmo, mas uma definição simples e de fácil entendimento é:

Conceito

“Conjunto de passos finitos que visam atingir um objetivo”.



No nosso dia a dia executamos algoritmos a todo o tempo. Vejamos um exemplo dos passos para o algoritmo “Acordar”:

1. Desligar o despertador do celular;
2. Levantar da cama;
3. Caminhar até o banheiro;
4. Abrir a torneira da pia;
5. Jogar água no rosto;
6. Fechar a torneira;
7. Passar creme dental na escova;
8. Escovar os dentes (Aqui poderíamos ter outro algoritmo para este passo);

Percebe-se que, em alguns passos, podem ocorrer fatos que não eram esperados, como por exemplo, não haver pasta dental ou não haver água na torneira. Nestes casos, outros algoritmos deveriam ser executados.

Tarefas repetitivas também são candidatas a serem modeladas através de algoritmos. Vamos supor que você tenha que inserir um prego na parede do escritório para pendurar o seu diploma do curso. Para realizar esta tarefa, você tem que segurar o prego sobre a parede e bater com o martelo tantas vezes quantas forem necessárias até que o prego fique firme e suporte o quadro com o seu diploma. Uma solução para este problema seria:

- 1) Repetir a seguinte sequência de ações:
 1. Segurar o prego sobre a parede com a mão esquerda;
 2. Bater com o martelo no prego, com a mão direita;
 3. Verificar se o prego já está no ponto certo.

Figura 02: Sequência de Ações

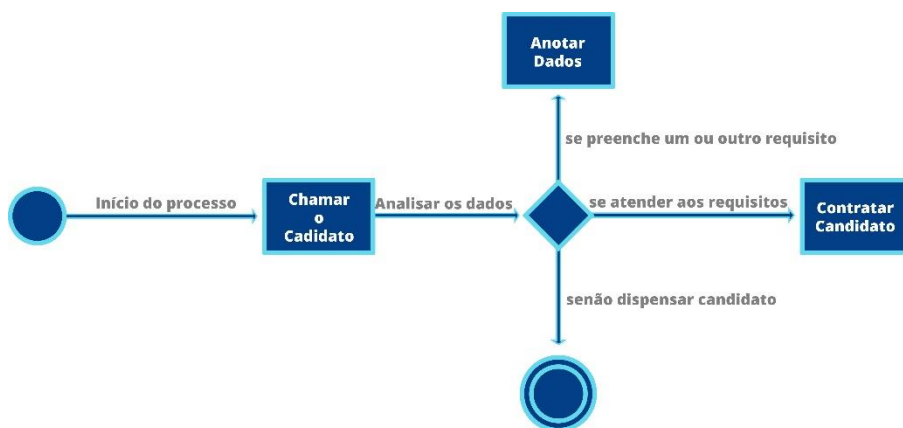


Fonte: iStock.com

Vamos supor que você precise realizar uma seleção de candidatos para o cargo de Analista de Sistemas na sua empresa, e há dois requisitos a serem preenchidos. Você deverá contratar os que preencherem os dois requisitos, anotar os dados de quem preencher apenas um dos requisitos e dispensar os que não preencherem nenhum dos dois requisitos. Poderíamos escrever uma solução para este problema da seguinte forma:

- 1º - chamar o candidato;
- 2º - se preencher os dois requisitos, então, contratar;
- 3º - caso contrário, se preenche um ou outro requisito, então, anotar seus dados;
- 4º - senão, dispensá-lo.

Figura 03: Representação do processo através do Diagrama de Atividades da UML



Fonte: Design EaD

O que se pode observar nessa solução é que está sendo feita uma seleção do candidato que satisfaz as duas condições (requisitos) para o cargo, e também uma segunda seleção (possivelmente para uso futuro) dos candidatos que satisfazem apenas uma das condições.

Essas seqüências de passos, que resolvem cada um dos exemplos propostos, é o que pode ser chamado de algoritmo para a solução do problema. Elas exemplificam as três formas básicas de se definir uma solução para qualquer problema:

1. Sequência.
2. Repetição.
3. Seleção.

Naturalmente que os algoritmos para problemas resolvidos com o auxílio de computadores não serão tão simples e nem apresentados como os exemplos mostrados.

O princípio central da programação é o conceito de algoritmos, isto é, programar é basicamente construir algoritmos. Verifique os principais pontos na conceituação de Algoritmo:



- É a descrição, de forma lógica, dos passos a serem executados no cumprimento de determinada tarefa.
- Serve como modelo para programas, pois sua linguagem é intermediária à linguagem humana e às linguagens de programação, sendo então, uma boa ferramenta na validação da lógica de tarefas a serem automatizadas.
- Como qualquer modelo, um algoritmo é uma abstração da realidade. A abstração é o processo de identificar as propriedades relevantes do fenômeno que está sendo modelado.
- Usando o modelo abstrato, podemos nos centrar unicamente nas propriedades relevantes para nós, dependendo da finalidade da abstração, e ignorar as irrelevantes.
- É a forma pela qual descrevemos soluções de problemas do nosso mundo, a fim de serem implementadas utilizando os recursos do mundo computacional. Existem regras básicas na forma de solucionar os problemas para que possamos utilizar os recursos de hardware e software disponíveis.
- Assim como executamos algoritmos no nosso cotidiano, podemos criar algoritmos para solucionar problemas computacionais ou informatizar processos das empresas e até mesmo da nossa vida.

As definições acima mostram que um algoritmo, para ser assim definido, precisa:

1. Ter início e fim.
2. Ser descrito em termos de ações não ambíguas e bem definidas.
3. Que as ações sigam uma sequência ordenada.



Sendo assim, um algoritmo é uma sequência finita de passos que levam a execução de uma tarefa para solução de um problema. Podemos pensar em algoritmo como uma receita, uma sequência de instruções que possui um objetivo específico. Estas tarefas não podem ser redundantes nem subjetivas na sua definição, devem ser claras e precisas, ou seja, um algoritmo deve ser uma sequência lógica de eventos, sem redundância ou dúvidas.

Como exemplos de algoritmos, podemos citar os algoritmos das operações básicas (adição, multiplicação, divisão e subtração) de números reais decimais. Outros exemplos seriam os manuais de aparelhos eletrônicos, como um smartphone, que explicam passo a passo as diversas funcionalidades.

Até mesmo as coisas mais simples podem ser descritas por sequências lógicas. Por exemplo, o algoritmo “Efetuar login em um Notebook”:

Figura 04: Algoritmo para “Efetuar login em um notebook”



Fonte: O autor

1.3 Algoritmo x Programa de Computador

Um algoritmo é uma sequência lógica de ações para se executar uma determinada tarefa. Um Programa é a formalização de um algoritmo em uma determinada linguagem de programação, segundo suas regras de sintaxe e semântica, de forma a permitir que o computador possa entender a sequência de ações (BUFFONI, 2003); (GUEDES S. , 2014).

Informalmente, um algoritmo é qualquer procedimento computacional bem definido que toma algum valor ou conjunto de valores como entrada e produz algum valor ou conjunto de valores como saída. Portanto, um algoritmo é uma sequência de passos computacionais que transformam a entrada na saída (CORMEN, LEISERSON, RIVEST, & STEIN, 2012).

Uma linguagem de programação é uma notação formal para descrição de algoritmos que serão executados por um computador. Como todas as notações formais, uma linguagem de programação tem dois componentes: **Sintaxe e Semântica**.

A sintaxe consiste em um conjunto de regras formais, que especificam a composição de programas a partir de letras, dígitos, e outros símbolos. As regras de semântica especificam o “significado” de qualquer programa, sintaticamente válido, escrito na linguagem (CORMEN, LEISERSON, RIVEST, & STEIN, 2012).

Os programas de computadores nada mais são do que algoritmos escritos numa linguagem de computador (Pascal, C, Java, C#, C++, entre outros) e que são interpretados e executados por uma máquina, no caso um computador (MEDINA & FERTIG, 2005).

Notem que dada esta interpretação rigorosa, um programa é, por natureza, muito específico e rígido em relação aos algoritmos da vida real.

Embora seja teoricamente possível a construção de computadores especiais, capazes de executar programas escritos em uma linguagem de programação qualquer, os computadores existentes hoje em dia são capazes de executar somente programas em linguagem de baixo nível, a Linguagem de Máquina (BUFFONI, 2003).

As Linguagens de Máquina são projetadas levando-se em conta os seguintes aspectos:

- Rapidez de execução de programas.
- Custo de sua implementação.
- Flexibilidade com que permite a construção de programas de nível mais alto.

Por outro lado, linguagens de programação de alto nível são projetadas em função de:

- Facilidade de construção de programas;
- Confiabilidade dos programas.

O **Problema** é: Como a linguagem de nível mais alto pode ser implementada em um computador, cuja linguagem é bastante diferente e de nível mais baixo?

Solução

Através da tradução de programas escritos em linguagem de alto nível para a linguagem de baixo nível do computador.

Para isso existem três tipos de programas tradutores: Montadores, Interpretadores e Compiladores. Os termos compiladores e interpretadores foram apresentados anteriormente, já “montador” pode ser considerado um sinônimo para a linguagem assembler, que traduz, através de mnemônicos (LOAD, STORE, MOVE, COMPARE etc.), os comandos equivalentes em linguagem de máquina (binária).

1.4 Construindo algoritmos

Os algoritmos são descritos em uma linguagem chamada pseudocódigo (MEDINA, 2005, p. 22). Esse nome é uma alusão à posterior implementação em uma linguagem de programação, ou seja, quando formos programar em uma linguagem, por exemplo: C, estaremos gerando código em C. Por isso, os algoritmos são independentes das linguagens de programação. Ao contrário de uma linguagem de programação, não existe um formalismo rígido de como deve ser escrito o algoritmo.

Figura 05: Construindo Algoritmos



Fonte: iStock.com

O algoritmo deve ser de fácil interpretação e codificação. Ele deve ser o intermediário entre a linguagem falada e a linguagem de programação.

Para construir um algoritmo, os seguintes passos devem ser observados (GUEDES S. , 2014):

1. Entender o problema a ser resolvido e destacar os pontos mais importantes e os objetos que o compõem;
2. Definir os dados de entrada, ou seja, quais dados serão fornecidos e quais objetos fazem parte do cenário do problema;
3. Definir o processamento, ou seja, quais operações serão efetuadas e quais as restrições necessárias;

4. Definir os dados de saída, ou seja, quais dados serão gerados após o processamento;
5. Construir o algoritmo através de uma das formas que serão apresentadas a seguir;
6. Testar o algoritmo realizando simulações, geralmente é utilizado o “teste de mesa”, ou seja, a simulação através de valores distintos nas entradas e análise das saídas.
7. Corrigir os erros e voltar ao passo 5.

O exemplo a seguir apresenta um algoritmo computacional para escolha e apresentação do maior valor entre dois números distintos introduzidos pelo usuário:



{Exemplo ALG01 - Algoritmo para escolha do maior de dois números}

Algoritmo "VerificaMaiorNumero"

var NUM1, NUM2, MAIOR: inteiro

inicio

escreva ("Digite o primeiro número:")

leia (NUM1)

escreva ("Digite o segundo número:")

leia (NUM2)

se (NUM1 > NUM2) entao

MAIOR <- NUM1

senao

MAIOR <- NUM2

fimse

escreva ("O maior número é ", MAIOR)

fimalgoritmo

I.5 Tipos de algoritmos

O desenvolvedor pode utilizar diversas formas para apresentar o problema a ser resolvido através de um algoritmo, sendo as principais:

- Descrição narrativa – consiste em descrever, em linguagem natural, o problema a ser resolvido, tendo como vantagem de que não é necessário nenhum novo conceito. A desvantagem da descrição narrativa é que pode ser interpretada de diversas formas.
- Fluxograma – consiste em descrever o problema através de recursos gráficos, como elipses, retângulos e setas. A vantagem é que imagens são mais simples de serem entendidas e a desvantagem está relacionada à dificuldade de se representar detalhes que deverão ser implementados em uma linguagem de programação.
- Pseudocódigo/portugol – consiste em representar o problema por meio de regras e uma estrutura pré-definida. A grande vantagem é que o algoritmo estará bem próximo da linguagem de programação que será utilizada. A desvantagem é uma compreensão profunda da lógica de programação e das regras do pseudocódigo.

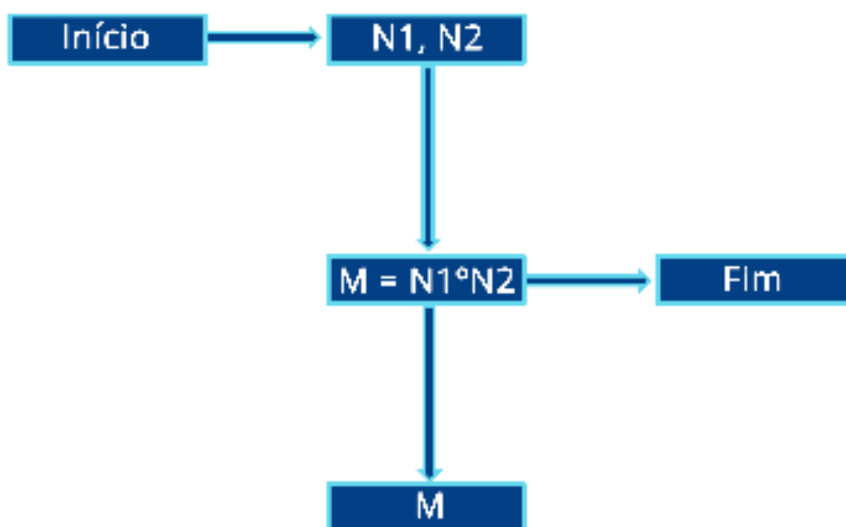
Exemplo de cada tipo de algoritmo para o problema de multiplicar dois números e apresentar o resultado.

Descrição narrativa

1. Receber dois números que serão multiplicados;
2. Multiplicar os números recebidos;
3. Apresentar o resultado.

Fluxograma:

Figura 06: Fluxograma



Fonte: Design EaD

Pseudocódigo/portugol

{Alg02 – Multiplica dois números}

Algoritmo "MultiplicaDoisNumeros"

Var M, N1, N2: inteiro

Inicio

 escreva ("Digite dois números: ")

 leia (N1)

 leia (N2)

$M \leftarrow (N1 * N2)$

 escreva ("O resultado da multiplicação é: ", M)

FimAlgoritmo

1.6 Exemplos de Algoritmos

Como visto anteriormente, um algoritmo pode ser representado de várias formas e para que possamos desenvolver algoritmos coesos, de fácil entendimento, lembre-se sempre de seguir os seguintes passos:



- Usar somente um verbo por frase;
- Imaginar que você está desenvolvendo um algoritmo para pessoas que não trabalham com informática;
- Usar frases curtas e simples;
- Seja objetivo;
- Procurar usar palavras que não tenham sentido dúbio.

Imagine o seguinte problema: Calcular a média final dos alunos da 3ª Série. Os alunos realizarão quatro provas: P1, P2, P3 e P4. Em que a média é calculada pela fórmula $((P1 + P2 + P3 + P4) / 4)$.

Para montar o algoritmo proposto, faremos três perguntas:

a) Quais são os dados de entrada?

R: Os dados de entrada são P1, P2, P3 e P4.

b) Qual será o processamento a ser utilizado?

R: O procedimento será somar todos os dados de entrada e dividi-los por 4 (quatro).

c) Quais serão os dados de saída?

R: O dado de saída será a média final.

O algoritmo em pseudocódigo:

```
{Alg03 - Calcula a média de quatro notas}
```

```
Algoritmo "MediaDasNotas"
```

```
var P1, P2, P3, P4: Inteiro
```

```
    R : real
```

```
inicio
```

```
    escreva ("Digite as quatro notas: ")
```

```
    leia (P1, P2, P3, P4)
```

```
    R <- ((P1 + P2 + P3 + P4) / 4)
```

```
    escreva ("A Média das notas é:", R)
```

```
FimAlgoritmo
```

I.7 Teste de Mesa

Após desenvolver um algoritmo, ele deverá sempre ser testado. Este teste é chamado de Teste de Mesa, que significa: seguir as instruções do algoritmo de maneira precisa para verificar se o procedimento utilizado está correto ou não.

Veja um exemplo:

Nota da Prova 1

Nota da Prova 2

Nota da Prova 3

Nota da Prova 4

Lançando valores diferentes para cada entrada, faça o processamento e verifique se o resultado está correto:

Figura 07: Teste de Mesa

P1	P2	P3	P4	Média
5	6	5	6	5,5
10	10	10	10	10
8	9	7	8	8
7	8	6	7	7

Fonte: O autor

No teste de mesa simulamos as entradas de dados que poderão ser realizadas pelo usuário, verificando assim a consistência da lógica desenvolvida no algoritmo.



II

Unidade II - Fundamentos de Lógica de Programação



Objetivos da Unidade

- ✓ Conhecer e realizar declaração de variáveis;
- ✓ Analisar os diferentes tipos de dados;
- ✓ Conhecer e aplicar os conceitos da álgebra das proposições.



Plano de Estudos

- ✓ Ciclo 02
- ✓ Atividade Exercício Individual
- ✓ Título: Variáveis e Declaração de Tipos

2. Introdução

Nesta unidade estudaremos as estruturas básicas de dados e a definição de tipos. Em seguida, serão apresentados os conceitos de constantes e variáveis. Para iniciar, faça uma boa leitura sobre a Álgebra das Proposições, que é essencial para o bom entendimento e construção de Algoritmos.

2.1 Introdução à lógica de programação

A lógica de programação é necessária para pessoas que desejam trabalhar com desenvolvimento de sistemas e programas, ela permite definir a sequência lógica para o desenvolvimento de um sistema computacional (CORMEN, LEISERSON, RIVEST, & STEIN, 2012).

Conceito

“Lógica de programação é a técnica de organizar um conjunto de procedimentos lógicos em uma estrutura que atinja um objetivo”

Estes procedimentos podem ser descritos como uma sequência de instruções, que devem ser seguidas para se cumprir uma determinada tarefa. Sequência Lógica são passos executados até atingir um objetivo ou solução de um problema.

Na linguagem comum, entende-se por instruções “um conjunto de regras ou normas definidas para a realização ou emprego de algo”. Em informática, porém, **instrução** é a informação que indica a um computador uma ação elementar a executar.

Convém ressaltar que uma ordem isolada não permite realizar o processo completo, para isso é necessário um conjunto de instruções colocadas em ordem sequencial lógica.



Por exemplo, se quisermos sacar dinheiro no caixa eletrônico, precisaremos realizar uma sequência de passos até atingir o objetivo final, sacar o dinheiro.

É evidente que essas instruções têm que ser executadas em uma ordem adequada – não se pode solicitar o saque sem antes autenticar a conta em que será realizado o saque.

Dessa maneira, uma instrução tomada em separado não tem muito sentido; para obtermos o resultado, precisamos colocar em prática o conjunto de todas as instruções, na ordem correta.

2.2 Tipos de Dados

Conceito

Um **tipo de dado** pode ser entendido como os possíveis valores que uma determinada variável poderá receber durante a execução de um programa.



Uma variável declarada com o tipo de dado numérico só poderá receber valores numéricos inteiros, positivos ou negativos em uma determinada faixa numérica.

Os termos mais comuns em programação e na construção de algoritmos são: **Variáveis e constantes**.

Uma **constante** é aquele valor que não se altera durante a execução de um programa de computador; mesmo que esse valor seja referenciado diversas vezes, o seu valor inicial deve ser mantido.

Já a **variável**, como o próprio nome sugere, modifica-se ao longo da execução de um programa, sendo que a cada momento ela pode receber ou fornecer valores diferentes.



Atente para o fato de que o valor das variáveis em um programa de computador, quase sempre, é armazenado em memória RAM (random access memory), a memória RAM é volátil, ou seja, depende de energia para manter os dados armazenados, caso o computador desligue, os dados das variáveis do programa serão perdidos. Não confunda os dados das variáveis com dados que estão armazenados em um Sistema Gerenciador de Banco de Dados (SGBD), são coisas diferentes que ao longo do curso ficarão bem claras.

Ao declararmos variáveis em qualquer programa de computador, precisamos definir duas informações básicas, o nome da variável e o tipo desta variável, sendo que é de extrema importância a definição correta deste tipo, pois é ele quem vai informar quais os possíveis valores que a sua variável poderá receber.



Ao definir uma variável chamada salário do tipo texto, estou informando que essa minha variável poderá receber todos os caracteres possíveis, porém não poderão ser realizados cálculos com ela, uma vez que não se faz nenhum tipo de cálculo matemático com texto.

Todas as variáveis devem assumir um determinado tipo de informação. Os tipos pré-definidos para algoritmos podem variar e, por convenção, vamos utilizar os seguintes:



Numéricos: destinam-se a armazenar valores ou constantes numéricas.

Literais: armazenam conjuntos de caracteres.

Lógico: armazenam apenas dois valores Falso ou Verdadeiro.

Conceito

Uma **constante** é um determinado valor fixo que não se modifica ao longo do tempo, durante a execução de um programa.

Uma constante pode ser um número (como se conhece na matemática), um valor lógico ou uma sequência de caracteres quaisquer com algum significado para o problema em estudo.

Conforme o seu tipo, a constante é classificada como sendo numérica, lógica ou literal.



Um exemplo clássico de constantes é o valor do PI, aquele número matemático que vale 3,1415... ao definirmos uma constante com esse valor, sabemos que ela nunca poderá ser alterada no programa.

Aprendemos, no ensino básico, que a fórmula para se calcular a área de uma circunferência é dada por $A = \pi \cdot r^2$. Tendo como base essa informação, vamos escrever o nosso algoritmo que calcula essa área.



```
{Alg04 - Calcula Raio}
Algoritmo "CalculaRaio"
var
{Variáveis}
```

```
    r : inteiro
    area : real
```

```
inicio
```

```
    {Constante}
```

```
    PI <- 3.1415
```

```
    escreva ("Insira o valor do raio da circunferência: ")
```

```
    leia (r)
```

```
    se (r > 0) entao
```

```
        area <- PI * r * r;
```

```
        escreva ("A área de circunferência do raio", r, " é ", area)
```

```
    senao
```

```
        escreva ("Raio menor ou igual a zero não é válido!")
```

```
    fimse
```

```
fimalgoritmo
```

Explicando o algoritmo:

Inicialmente são declaradas as variáveis r e área, no início do algoritmo declaro a constante PI valendo 3,1415. O comando “ESCREVA” solicita uma entrada de dados por parte do usuário que será armazenada na variável “raio”, logo abaixo através do comando “LEIA”. Na sequência, o valor informado pelo usuário é testado, caso seja maior que zero a área é calculada e escrita na tela, caso seja menor que zero a mensagem “Raio menor ou igual a zero não é válido!” é apresentada.

2.3 Tipos de constantes

Há vários tipos de constantes que podem ser declaradas em um algoritmo, cada uma possui características específicas.

Constante Numérica (Real ou Inteiro): A representação de uma constante numérica nos algoritmos é feita no sistema decimal, podendo ser um número com ou sem parte fracionária.

A parte decimal deve ser separada usando o ponto decimal e não a vírgula, como se tem costume. Se necessária a representação na forma exponencial, usamos a letra E para indicar essa situação.



Constantes do tipo Real são utilizadas em números que terão casas decimais, já as inteiras não.

a) 25 b) 3.14 c) $7.8 * 10^3$ d) $0.321E+02$



A constante numérica pode ser positiva ou negativa, de acordo com o sinal que precede os algarismos formadores do número. Caso não exista um sinal, a constante é considerada positiva. Veja abaixo:

a) 15 b) +15 c) -15 d) 0.342
e) -2.726 f) $9.7 * 10^6$ g) $-2.5 * 10^3$ h) $2.5 * 10^3$
i) $-0.91 * 10^{-9}$ j) 1041 l) -106 m) -10^{-9}

Constante Lógica: É um valor lógico, isto é, que só pode ser falso ou verdadeiro, usado em proposições lógicas. Só existem 2 constantes deste tipo, sendo representadas por FALSO ou VERDADEIRO ou então V ou F ou também 0 ou 1.

Constante Literal (Caracter ou String): Uma constante desse tipo pode ser qualquer sequência de caracteres (letras, dígitos ou símbolos especiais) que forme um literal com algum significado para o problema em estudo.

Podemos compor um literal usando letras, números e caracteres especiais como *, \$, # @, (e qualquer outro que conste na tabela ASCII). Toda constante literal que aparece no algoritmo será colocada entre aspas " " para que não seja confundida com outro item qualquer.



- a) "José da Silva" b) "Mensagem"
c) "123,45" d) "XIY2W3"
e) "*A!B?-" f) "23/05/55"



Note que um numeral encontrado entre "aspas" é considerado como uma sequência de dígitos (literal), e não como uma constante numérica. Também não se deve confundir uma constante lógica (por exemplo, falso) com uma sequência de caracteres (literal), que apareça entre aspas (por exemplo, "Falso").

2.4 Declaração de variáveis

Um dos primeiros passos rumo à criação de um algoritmo é a declaração das variáveis que serão utilizadas para armazenar as informações. Na maioria das linguagens de programação, a declaração das variáveis precede qualquer código da lógica.

Conceito

As **variáveis** podem ser consideradas informações que são armazenadas em memória principal. Realizando uma comparação entre a memória principal de um computador com um armário em que são armazenadas informações, as gavetas seriam as variáveis em que diferentes valores poderão ser colocados e retirados a todo instante (GUEDES S., 2014).



Em outras palavras, uma variável declarada em um programa de computador é a reserva de um espaço na memória RAM, sendo que a cada execução do programa este local pode ser alterado de acordo com o uso da memória. O tamanho do espaço que será alocado para a variável dependerá do tipo que essa variável foi declarada, como por exemplo: real, numérico, caracter, etc.

Cada linguagem de programação possui os seus próprios tipos de dados que podem ser atribuídos a variáveis, sendo que cada uma aloca um tamanho diferente de espaço em memória. A documentação de cada linguagem especifica detalhadamente o tamanho que será alocado em memória.



Algumas linguagens de programação atuais possuem um recurso chamado de “tipagem dinâmica”, em que não se declara o tipo da variável, o tipo é definido de acordo com o valor que a variável recebe durante a execução do programa. Apesar de parecer uma forma bastante interessante e que pode diminuir erros na programação, a “tipagem dinâmica” traz um esforço computacional bem maior, pelo fato de que o compilador ou interpretador necessita verificar e avaliar os dados antes de realizar a atribuição à variável. O ideal é que este recurso seja usado com cautela pelo programador e que faça uma utilização híbrida da declaração estática e da dinâmica.

Uma variável possui, basicamente, duas partes: a primeira parte é o seu **nome** (identificador), que deverá seguir algumas regras, como veremos a seguir, e a segunda é o seu **tipo**.

Formação dos Identificadores (nomes das variáveis): um identificador é formado por 1 ou mais caracteres, sendo que o 1º caractere deve, obrigatoriamente, ser uma letra e os caracteres seguintes, letras ou dígitos, não sendo permitido o uso de símbolos especiais. O Identificador é definido por quem está escrevendo o algoritmo obedecendo alguns critérios assim convencionados (GUEDES S. , 2014):

- As letras podem ser maiúsculas ou minúsculas, sendo recomendadas letras maiúsculas;

- O primeiro caracter deve ser letra ou `_`. Não pode ser um número.
- Não são aceitos caracteres especiais como `@`, `.`, `/`, `+`, etc., com exceção do `_` (underscore).
- Caracteres com acento são considerados especiais, como por exemplo: `é`, `á`, `ó`, `í`, `ú`, `ã`, `ç`, etc.
- Espaços em branco também não são permitidos.
- Não devemos fazer uso de palavras-chave, isto é, palavras que terão um significado próprio no algoritmo como os comandos, funções, etc. As principais palavras-chave (palavras reservadas) são: `algoritmo`, `fim algoritmo`, `declare`, `leia`, `escreva`, `se`, `então`, `senão`, `numérico`, `literal`, `lógico`, `repita`, `fim repita`, `interrompa`.

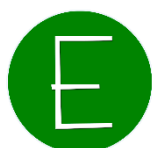
É recomendável que os nomes das variáveis sejam os mais significativos possíveis, isto é, que reflitam da melhor maneira a natureza dos valores que nelas estão sendo armazenados. Isso ajuda muito no entendimento do algoritmo.



Se a variável vai armazenar o salário de um empregado, porque não escolher o identificador `SALARIO` para representá-la e o tipo inteiro?

Alguns exemplos de nomes de identificadores são apresentados a seguir:

a) Permitidos:



NOTA MATRICULA NOMEALUNO nota001

MEDIA_NOTA NOMEALUNO002 nomealuno02

b) Não permitidos:

01ALUNO X-Y NOTA[I] ^nota

ALUNO-MATRICULA NR.FILHOS

Após definido o identificador da variável, precisamos realizar a declaração no algoritmo, para isso algumas regras devem ser seguidas.

Toda declaração de variáveis tem a seguinte forma:



```
var lista_de_identificadores : nome_do_tipo
```

Sendo:

- **declare:** é uma palavra-chave, ou reservada, do algoritmo que deve ser sempre utilizada para declarações.
- **Lista_de_identificadores:** são os nomes escolhidos para as variáveis, que devem estar separadas por vírgula.
- **Nome_do_tipo:** é uma das três palavras-chave, numérico, literal ou lógico que indicam o tipo associado às variáveis. Observar que após o tipo deve ser inserido o ponto e vírgula.

- Atente para o “:” entre o nome da variável e o nome do tipo. Alguns livros não utilizam esta notação, mas no nosso caso vamos adotá-la.



```
var NOTA, CODIGO : inteiro
    TESTE, SIM : logico
    NOME, ENDERECO caractere
```



Lembre-se que palavras-chave não podem ser utilizadas como identificadores de variáveis, as regras apresentadas aqui valem para qualquer tipo de linguagem de programação, trata-se de um “design pattern” ou padrão de desenvolvimento.

A seguir são apresentados vários exemplos de declaração de variáveis.

Exemplo I: Calcular e escrever a média aritmética de dois números quaisquer.

```
Algoritmo “NomeAlgoritmo”
    var N1, N2, MEDIA : inteiro
inicio
    {desenvolvimento do algoritmo}
finalgoritmo
```

Exemplo 2: Faça um algoritmo que calcule a média das quatro notas semestrais de um aluno. Após o cálculo, apresentar o nome e a média final do aluno.

```
Algoritmo "NomeAlgoritmo"
var NOME caractere
    N1, N2, N3, N4, MEDIA : inteiro
inicio
    {desenvolvimento do algoritmo}
finalgoritmo
```

Exemplo 3: Resolver uma equação do 2º grau. Sabemos que uma equação do 2º grau tem a forma $ax^2 + bx + c = 0$, $a \neq 0$. Sabemos, também, que uma equação do 2º grau tem no máximo duas raízes, que variam de equação para equação e que para resolvê-la precisamos determinar o (delta). Logo, apresentamos todas as variáveis necessárias.

```
Algoritmo
declare A, B, C, RAIZ1, RAIZ2, DELTA : numérico;
fim algoritmo
```



Observe que o símbolo " Δ ", embora tenha um significado claro para nós na matemática, não foi usado como nome de variável. Por quê?



Nesses exemplos estamos apenas identificando as entradas (declarações), nos próximos tópicos iremos detalhar o processamento e a saída (desenvolvimento do algoritmo).

2.4.1 Comentários em algoritmos

Com a finalidade de simplificar a compreensão de um algoritmo, ou seja, torná-lo claro e de fácil entendimento, é utilizado um instrumento denominado comentário. Ele é um texto ou simplesmente uma frase, que aparece sempre delimitado por chaves.

Forma geral:

{o texto que você deseja acrescentar}



Algoritmo "NomeAlgoritmo"

var NUM1, {primeiro número a ser somado}

NUM2, {segundo número a ser somado}

S {soma dos dois números} : inteiro

inicio

{ler os números}

{calcular sua soma}

{escrever a soma}

finalgoritmo

Os comentários podem ser colocados em qualquer ponto do algoritmo onde se façam necessários. No exemplo a seguir, é demonstrado um conjunto de declarações onde foram introduzidos comentários com o intuito de explicar o significado de cada uma das variáveis.

Leia o nome e o salário de um funcionário. Escreva seu nome somente se seu salário for maior que R\$ 500,00.



```

Algoritmo "NomeAlgoritmo"
    var NOME : caractere {nome do funcionário}
    SAL : inteiro {salário}
    inicio
        {resolver o problema}
    finalgoritmo
    
```

Calcular e escrever os juros obtidos na aplicação de um capital aplicado a uma certa taxa por um determinado tempo.



```

Algoritmo "CalculoDeJuros"
    {declaração das variáveis}
    var C, {capital}
    I, {taxa}
    T, {tempo}
    JJ {juro} : inteiro
    inicio
        {ler os valores, calcular e escrever o juro}
    finalgoritmo
    
```

A declaração de variáveis é um dos primeiros passos na criação de um algoritmo, o desenvolvedor precisa atentar para o problema e identificar corretamente quais e de que tipo serão todas as variáveis que solucionarão o problema.

2.5 Softwares de apoio

O principal software utilizado no Brasil para o aprendizado de algoritmos é o **Visualg**¹. O VisuAlg é um programa que edita, interpreta e executa algoritmos com uma linguagem próxima do português estruturado como um programa normal de computador. É um programa de livre uso e distribuição, empregado no ensino de programação em várias escolas e universidades no Brasil e no exterior.

Uma outra possibilidade para criar e executar os seus programas, até mesmo no celular, é através do APP **AndroidVisual**, disponível gratuitamente na Play Store para dispositivos Android. O aplicativo utiliza a mesma sintaxe do Visualg, assim os seus algoritmos escritos no computador através do Visualg poderão ser executados também no aplicativo e vice-versa.

¹ Disponível em: <<http://sourceforge.net/projects/visualg30/>>. Acesso em: 19. Jan. 2016.



III

Unidade III – Operadores, expressões e comandos básicos



Objetivos da Unidade

- ✓ Conhecer os operadores, expressões e comandos básicos que podem fazer parte de um algoritmo. Nesta unidade, o objetivo principal é apresentar os diversos recursos relacionados à matemática, que certamente farão parte de qualquer programa de computador, visto que utilizam, basicamente, a lógica e seus princípios.



Plano de Estudos

- ✓ Ciclo 03
- ✓ Atividade Exercício em Grupo
- ✓ Título: Aplicando a Lógica na Prática

3. Introdução

Nesta unidade serão apresentados os conceitos de expressões aritméticas e operadores, assim como novos comandos que permitirão o desenvolvimento de algoritmos completos.

3.1 Operadores

O conceito de operadores é fundamental para a compreensão e criação de algoritmos.

Conceito

Operadores representam expressões de cálculos, condição, atribuição, comparação e concatenação (junção). (GUEDES S. , 2014).

Os operadores possuem classificações, visando distingui-los de acordo com o uso e aplicação.

Operador de atribuição: representado em algoritmos pelos símbolos "<-". Utiliza-se a atribuição quando o programador deseja alocar valores para uma determinada variável.



```
Nome <- "Rodrigo F. Frogeri"
Resultado <- a + 10
Valor <- 10.00
```

Operadores aritméticos: são utilizados para a realização de cálculos matemáticos, ou seja, é um conjunto de símbolos para a realização das operações matemáticas.

Operadores	Representação na notação algorítmica
Incremento	Utiliza-se uma expressão. Exemplo: $a \leftarrow a + 1$
Decremento	Utiliza-se uma expressão. Exemplo: $a \leftarrow a - 1$
Multiplicação	*
Divisão	/
Exponencial	^ ou ** Exemplo: Pot $\leftarrow 2^{**}3$ Pot $\leftarrow 2^3$ Raiz $\leftarrow 4^{(1/2)}$
Módulo	Mod (é o resto de uma divisão) Exemplo: Resto $\leftarrow a \text{ mod } b$
Subtração	-

Operadores relacionais: este tipo de operador é utilizado para a realização de comparações entre expressões. Atente-se para o fato de que o resultado da utilização de um operador relacional será sempre um valor lógico, ou seja, sempre VERDADEIRO ou FALSO.

Operadores	Representação na notação algorítmica
Maior	>
Maior ou igual	>=
Menor	<
Menor ou igual	<=
Igual a	=
Diferente de	<>

Operadores lógicos: são utilizados para a realização de junções ou associações entre expressões. O resultado deste tipo de operador será sempre lógico (Verdadeiro ou Falso). Os operadores lógicos são três: E, OU e NÃO.

Operador	Representação na notação algorítmica	Outras notações
E (Junção)	e, and, E	&& ou ^
OU (Disjunção)	ou, OR, OU	ou v
NÃO (Negação)	não, NOT, NÃO	!



Para o correto entendimento da utilização dos operadores lógicos, precisamos ter em mente as seguintes observações: (1) Para que o resultado em junções com o operador E seja verdadeiro, TODAS as afirmações precisam ser verdadeiras. (2) Para que o resultado em disjunções do operador OU seja verdadeiro, apenas UMA deve ser verdadeira. (3) A negação apenas inverte a afirmação.



Têm-se duas afirmativas, $((9 = 10) \text{ E } (1 = 1))$. Podemos verificar que a primeira expressão resulta em FALSO e a segunda em VERDADEIRO; visto que está sendo utilizado o operador E, o resultado será FALSO, pois FALSO E VERDADEIRO = FALSO. Mas como saberemos todas as possibilidades?

Neste momento, surge a tabela verdade para nos ajudar. Essa tabela apresenta todos os possíveis resultados para expressões lógicas E, OU e NÃO.

Tabela 01: Tabela verdade para duas variáveis

A	B	A AND B	A OR B	NOT A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Fonte: O autor

A tabela verdade foi criada tendo como base apenas duas variáveis, A e B, sendo que é possível a comparação de um conjunto enorme de proposições, basta que as mesmas regras para duas variáveis sejam seguidas para três, quatro, cinco, etc.

Vejamos um exemplo: Dada a expressão lógica $((A \geq 5) \text{ AND } (B \neq 10))$, crie a tabela verdade correspondente, sendo que $A = 5$ e $B \neq 9$.

Tabela 02: Tabela verdade p/ $A = 5$ e $B \neq 9$

Expressão algoritmo	$A = 5$	$B \neq 9$	Operador		
			$\&\&$.e.	$\parallel$ (ou)	! (não) p/ $A = 5$
Resultados possíveis	V	V	V	V	F
	V	F	F	V	F
	F	V	F	V	V
	F	F	F	F	V

Fonte: Guedes (2014), p. 21.



Você pode estar se perguntando, mas como os valores VERDADEIRO e FALSO foram atribuídos para $A = 5$ e $B \neq 9$? Neste caso, precisam ser consideradas as possíveis entradas de dados e a quantidade de variáveis envolvidas. Se temos apenas duas variáveis, teremos $2^{\text{quantidade de variáveis}} = 4$, ou seja, quatro possibilidades de combinações. Considere as entradas de dados valendo $A = 5$ e $B = 10$, assim teremos a seguinte tabela verdade:

Tabela 03: Tabela verdade p/ $A = 5$ e $B = 10$

$A = 5$	$B \neq 9$	Resultado lógico de A	Resultado lógico de B	Resultado (AND)
5	10	V	V	V
5	9	V	F	F
4	10	F	V	F
4	9	F	F	F

Fonte: O autor

A utilização de operadores fará parte da vida de um profissional da área de tecnologia da informação, sendo que praticamente todos os problemas a serem resolvidos utilizarão, de alguma forma, esses operadores. Assim, a compreensão e capacidade de criação de tabelas verdades são de extrema importância.



Entenda, compreenda, revise e não deixe dúvidas sobre esse assunto. Como sugestão, você pode instalar e utilizar, no seu celular, o aplicativo para sistema operacional móvel Android "Truth Tables", que possibilita a geração de tabelas verdades com todos os operadores lógicos. Vale a pena conferir!

3.2 Expressões Aritméticas

Para que possamos utilizar expressões aritméticas, devemos lembrar o conceito de operador e de operandos, ou seja, a operação que será realizada e em que tipo de variável será aplicada.



Para que possamos utilizar expressões aritméticas, devemos lembrar o conceito de operador e de operandos, ou seja, a operação que será realizada e em que tipo de variável será aplicada.

Conceito

Denomina-se **expressão aritmética** aquela cujos operadores são aritméticos e cujos operandos são constantes e/ou variáveis do tipo numérico.

O conjunto de operações básicas adotado é o que se conhece da matemática: adição, multiplicação, potenciação, subtração, divisão e radiciação. Segue abaixo a tabela com os operadores.

Tabela 04: Operadores aritméticos

Operadores aritméticos	Símbolo
Potenciação	$\wedge$
Produto	$\times$
Divisão	$/$
Soma	$+$
Subtração	$-$

Fonte: O autor

	a) $X + Y$ c) $2 * \text{NOTA}$ e) $A * B * C$	b) $X - Y$ d) TOTAL/N
--	--	-----------------------------------

A notação utilizada para expressão aritmética nos algoritmos é basicamente a mesma da matemática, com exceção das seguintes restrições:

a) Não é permitido omitir o operador de multiplicação, o que é comum nas expressões matemáticas. Isto evita confusão quanto aos nomes de variáveis, pois numa expressão da forma $AB + C$, como saber se AB é nome de uma variável, ou a multiplicação entre os conteúdos de duas variáveis, cujos nomes são A e B ?

b) Nas expressões aritméticas, as operações guardam entre si uma relação de prioridade, tal como na matemática:

Tabela 05: Relação de Prioridade dos Operadores Aritméticos

Prioridade	Operação
1 ^a	Potenciação, radiciação
2 ^a	Multiplicação, divisão
3 ^a	Adição, subtração

Fonte: O autor



Os parênteses são os únicos sinais de agrupamento e são capazes de alterar a hierarquia das operações; não se admitem chaves ou colchetes.



A expressão $A + B * C^2$ será diferente de $((A + B) * (C^2))$, pois a ordem de cálculo da primeira será: Potência, multiplicação e soma. Já na segunda será: Parênteses 1, parênteses 2 e depois a multiplicação.

Para se obter uma sequência de cálculo diferente, vários níveis de parênteses podem ser usados para quebrar as prioridades definidas. Não é permitido o uso de colchetes ou chaves, uma vez que estes símbolos são utilizados nos algoritmos para outras finalidades.

3.3 Funções aritméticas

Além das operações básicas anteriormente citadas, pode-se usar nas expressões aritméticas algumas funções muito comuns na Matemática. São funções que a maioria de nós já utilizou nas calculadoras científicas e, sem elas, ficaria impossível fazer alguns cálculos mais complicados.

A tabela a seguir fornece a sintaxe que usaremos nos Algoritmos. O termo EA significa Expressão Aritmética e pode ser um número constante, uma variável ou uma expressão matemática qualquer.

Tabela 06: Principais expressões aritméticas

Função	Resultado	Tipo do resultado
LOG (EA)	Logaritmo na base 10 de EA	Real
LN (EA)	Logaritmo Neperiano de EA	Real
EXP (EA)	Exponencial de EA	Real
ABS (EA)	Valor Absoluto de EA	Real/Inteiro
TRUNCA (EA)	Parte inteira de EA	Inteiro
ARREDONDA (EA)	Arredondamento de EA	Inteiro
QUOCIENTE (EA, EB)	Quociente da divisão de EA por EB	Inteiro
RESTO (EA, EB)	Resto da divisão de EA por EB	Real
SEN (EA)	Seno de EA (EA dado em radianos)	Real
COS (EA)	Cosseno de EA (EA dado em radianos)	Real
TAN (EA)	Tangente de EA (radianos)	Real

Fonte: O autor



- 1) A função ABS(EA) fornece resultado do tipo Real quando o argumento é uma expressão do tipo Real, e Inteiro quando o argumento é do tipo Inteiro.
- 2) Através das funções SEN, COS E TAN, pode-se criar qualquer outra função trigonométrica.

A função atua sobre um argumento numérico, que é o resultado obtido após a avaliação da expressão aritmética entre parênteses. As palavras que designam as funções são escritas com letras maiúsculas.



- a) $X + \text{SEN}(A + B + C)$ b) $\text{QUOCIENTE}(\text{NOTA}, 2) * 100 + T$
 c) $X + \text{LN}(Y) - \text{ABS}(A - B)$ d) $H2 - G * F * \text{RESTO}(C / D)$

3.4 Prioridades

Como foi visto, pode-se ter mais de um operador lógico na mesma expressão. Em alguns casos, dependendo dos valores envolvidos, a ordem em que são efetuadas as operações lógicas afeta o resultado final. Assim, como acontece entre as operações aritméticas, também existe uma relação de prioridade entre os operadores lógicos.

Na tabela abaixo são apresentadas as prioridades entre os operadores conhecidos, visto que podem estar presentes na mesma expressão lógica.

Tabela 07: Prioridades entre as expressões

Prioridades	Operadores	Símbolos
1º	Aritméticos	+, -, *, /
2º	Relacionais	>, >=, <, <=, <>
3º	Negação	Não
4º	Conjunção	E
5º	Disjunção	OU

Fonte: O autor

Assim como nas expressões aritméticas, nas expressões lógicas vários níveis de parênteses podem ser utilizados com a finalidade de estabelecer uma nova ordem de execução entre os operandos lógicos.



- a) $A = 1 \text{ e } (B + C <> 0 \text{ ou } K \leq 2)$
 b) não $(TOTAL \geq 2 \text{ e } A <> B)$ ou TESTE

No caso do exemplo A, tudo que está entre parênteses será executado primeiro, na seguinte ordem:



SOMA -> DIFERENÇA (<>), <=, OU. Resolvido o parêntese, será feita a igualdade de $A = 1$ e, por último, o "E".

A ordem de execução do exemplo B será a seguinte:



Parêntese, >=, <>, E, NÃO, OU).

3.5 Comando de Atribuição

Conceito

Definimos “comando” como sendo uma ordem que desejamos que seja cumprida em determinado momento do algoritmo. Um conjunto de comandos é encarregado de descrever a lógica necessária para que um problema seja resolvido com sucesso.

O comando de atribuição permite a atribuição de um valor a uma variável, desde que haja compatibilidade de tipos, ou seja, eu não posso atribuir um valor NUMÉRICO para uma variável que foi declarada como STRING. Isto significa que nunca será permitido atribuímos valores a variáveis que não suportem aquele tipo de dado.



Esse tipo de erro, geralmente, é apresentado pelos compiladores como “mismatch”, ou tipo incompatível.

Forma para atribuição:

variável <- variável ou constante ou expressão;

onde:

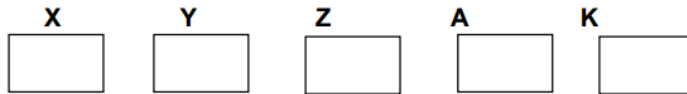
- **identificador:** é o nome da variável à qual está sendo atribuído o valor;
<- : é o símbolo de atribuição;
- **variável ou constante ou expressão.**

	a) K <- I; e) MEDIA <- SOMA / N; b) COR <- "VERDE"; f) COD <- N2 + >= 5; c) TESTE <- F; g) SIM <- X = 0 e Y <> 2;
---	---

Visualmente, sabemos que o comando “declare” abre posições na memória. Assim, o comando:

	<pre>declare X,Y, Z : numérico; A : literal; K : lógico;</pre>
---	--

O comando produzirá cinco espaços em memória de acordo com o tamanho do tipo da variável.



Os espaços estão reservados na memória, mas sem valores. Se, em seguida, tivéssemos os seguintes comandos de atribuição, nesta ordem:

X <- 56;

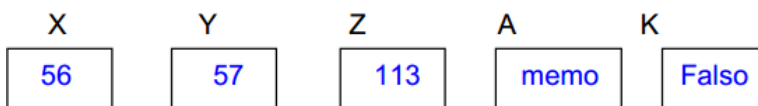
Y <- X+1;

Z <- X+Y;

A <- "memo";

K <- Falso;

O resultado na memória seria:



Visto que cada posição na memória corresponde a um único valor, se, em seguida, tivéssemos os seguintes comandos de atribuição:

X <- 20

K <- Verdadeiro

O valor de $X = 56$ foi perdido e só ficará armazenado o 20; o mesmo acontece com K que mudaria de Falso para Verdadeiro

X	Y	Z	A	K
20	57	113	memo	Verdadeiro



O que ocorrerá se tentarmos armazenar o valor "Falso" na variável K?



- ✓ À esquerda do sinal $\leftarrow$ só podemos ter uma variável;
- ✓ À direita podemos ter uma variável, uma constante ou uma expressão, desde que todas sejam do mesmo tipo da variável da esquerda do sinal $\leftarrow$
- ✓ O sinal $\leftarrow$ não tem aqui o sentido de igualdade, mas sim de atribuição; isto pode ser claramente exemplificado através do exemplo $X \leftarrow X + 1$; se $\leftarrow$ indicasse igualdade teríamos $X = X + 1$ ou $X - X = 1 \leftarrow 0 = 1$.
- ✓ Como $\leftarrow$ não representa sinal de igualdade, não poderá ser usado em expressão lógica; não será aceito, por exemplo: $(A \leftarrow B \text{ e } X > Y)$ e $\text{sim } (A = B \text{ e } X > Y)$.



Voltando à pergunta anterior, devemos atentar para o fato de que “Falso” é diferente de Falso, o primeiro é uma string ou um tipo texto, já o segundo é um valor booleano ou lógico, que aceita apenas Verdadeiro ou Falso e 0 ou 1. A atribuição realizada no exemplo causará um erro de tipo incompatível, ou “type mismatch”, como muitas linguagens de programação apresentam no compilador.

Considere o seguinte algoritmo:

Algoritmo “Alg05 - ExemploAtribuicao”

var X, Y, Z : inteiro

A, B, C : caractere

L, M : logico

inicio

X <- 23,45;

A <- “TESTE”

L <- Falso

Y <- X + 2

Z <- X + Y

M <- L

C <- “BEM”

B <- A

B <- “A”

X <- Z

fimalgoritmo



A saída do algoritmo anterior, no software Visualg, será:

Escopo	Nome	Tipo	Valor
GLOBAL	X	I	48
GLOBAL	Y	I	25
GLOBAL	Z	I	48
GLOBAL	A	C	TESTE
GLOBAL	B	C	A
GLOBAL	C	C	BEM
GLOBAL	L	L	FALSO
GLOBAL	M	L	FALSO

Lembrando que uma variável corresponde a uma única posição na memória, podemos observar que no final deste algoritmo as variáveis X, Y, Z, A, B, C, L e M estarão valendo, respectivamente, 48.90, 25.45, 48.90, "TESTE", "A", "BEM", Falso e Verdadeiro.

No exemplo anterior, não seriam válidas as atribuições:

- `X <- A`
- `B <- Falso`
- `L <- "verdadeiro"`



E muitas outras em que os tipos não fossem compatíveis.

3.6 Comandos de Entrada e Saída

Se o comando de atribuição fosse a única maneira de armazenarmos dados na memória do computador, teríamos uma série de dificuldades quando quiséssemos alterar o valor de uma variável.



Imagine um algoritmo que faz o cálculo de aumento de salário de um funcionário. Se tivéssemos que atribuir o valor do salário no algoritmo, a cada funcionário teríamos que alterar o algoritmo e executá-lo novamente, o que é inviável.

O ideal é que o computador nos permitisse digitar o salário desse funcionário e em seguida realizasse o armazenamento na memória principal; e também nos fornecesse o valor calculado. Isto pode ser feito usando os comandos de Entrada e Saída (input/output).

3.6.1 Comando de Entrada

Permite a introdução de dados na memória do computador, através de um dispositivo de entrada, como por exemplo: o teclado. O comando de entrada que irá ler os dados digitados no teclado é o “LEIA”.

Forma geral:

leia (lista de variáveis separadas por vírgula);

Em que **leia** é uma palavra-chave e “lista de variáveis” deve conter as variáveis citadas no declare.



A atribuição será feita após a entrada do dado e o pressionamento da tecla ENTER, assim uma nova informação para a próxima variável estará habilitada, sendo possível que várias variáveis sejam especificadas no comando LEIA.



Dados dois números quaisquer, calcular e escrever o seu produto.

Algoritmo "Alg06 - ExemploEntradaLeia"

Var A, B: inteiro {dois números quaisquer}

PROD : inteiro {produto dos dois números}

inicio

leia (A, B) {Aqui o usuário deverá digitar um número, pressionar ENTER e digitar o próximo, automaticamente cada entrada será atribuída para A e B}

PROD <- A * B {Os valores digitados são multiplicados e atribuídos a variável PROD}.

{escreva a resposta}

escreva ("O produto é ", PROD)

Fimalgoritmo



Leia o nome, idade, e salário de um funcionário. Calcule e escreva seu novo salário com um acréscimo de 10%.

Algoritmo "Alg07 - ExemploEntradaLeia2"

Var NOME : **caractere**
 IDADE, SAL : **inteiro**
 NOVO : **real**

inicio

leia (NOME, IDADE, SAL)
 NOVO <- SAL+10/100*SAL
 {escreva novo salário}
 escreva ("O novo salário é ", NOVO)

Fimalgoritmo



Se fôssemos descrever, passo a passo, o algoritmo anterior, como seria?



Variáveis literais quando recebem um valor numérico (0 a 9) como entrada não apresentam erro, pois consideram que este valor é um texto, ou seja, considera-se um número como um caracter. Já quando há a inserção de letras em variáveis que são numéricas, será disparada uma exceção de tipo incompatível (type mismatch).



Voltando à pergunta anterior, o algoritmo irá aguardar a entrada de três valores, sendo que o primeiro deverá ser literal e os demais do tipo numérico. Na sequência é realizado o cálculo do novo salário, tendo como base o valor da variável SAL.

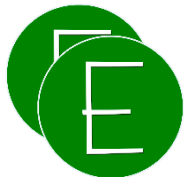
3.6.2 Comando de Saída

Como o objetivo do algoritmo é estabelecer regras e descrever passos que levem à solução de um problema, faz-se necessário um comando que permita a saída dos resultados obtidos na solução. Se a função do algoritmo é, por exemplo, calcular a média aritmética de dois números, esta média precisa ser apresentada. Isto se faz usando o comando **escreva**.

Forma geral:

escreva (lista de variáveis separadas por vírgula);

Em que “escreva” é uma palavra-chave e a “lista de variáveis” ou “constantes” são separadas por vírgula, que deve vir entre parênteses e terminar com o ponto e vírgula. Pode-se utilizar aspas duplas dentro do comando escreva para que textos personalizados sejam apresentados em conjunto com o valor da variável.



A seguir é apresentado o mesmo algoritmo do comando leia, porém com o comando escreva no código.

Algoritmo "Alg08 - ExemploEntradaLeia"

var A, B : inteiro {dois números quaisquer}

PROD : inteiro {produto dos dois números}

inicio

leia (A, B) {Aqui o usuário deverá digitar um número, pressionar ENTER e digitar o próximo, automaticamente cada entrada será atribuída para A e B}

PROD <- A * B {Os valores digitados são multiplicados e atribuídos a variável PROD}.

escreva ("O produto dos dois números é: ", PROD)

Fimalgoritmo

Algoritmo "Alg09 - ExemploEntradaLeia2"

```

Var    NOME : caractere
        IDADE, SAL : inteiro
        NOVO : real

inicio
    leia (NOME, IDADE, SAL)
    NOVO <- SAL + 10/100*SAL
    escreva ("O novo salário é de: ", NOVO)
Fimalgoritmo

```

Ao se deparar com o comando “leia”, o computador fica aguardando que os valores sejam digitados, sem dar uma mensagem. Consequentemente, a pessoa que está fazendo uso do programa escrito, a partir dos algoritmos apresentados, não será informada sobre o que o computador está aguardando, não sabendo o tipo de informação a ser informada.

Para que esse problema seja solucionado, o ideal é que mensagens que solicitam informações sejam apresentadas antes dos comandos de leitura, como a seguir:

Algoritmo “Alg10 – ExemploEntradaLeia”

```

Var    A, B : inteiro {dois números quaisquer}
        PROD : inteiro {produto dos dois números}

inicio
    escreva ("Informe o primeiro valor: ")
    leia (A)
    escreva ("Informe o segundo valor: ")
    leia (B)
    PROD <- A * B
    escreva ("O produto dos dois números é: ", PROD)
Fimalgoritmo

```



Algoritmo "Alg II - ExemploEntradaLeia2"

Var NOME : caractere

IDADE, SAL : inteiro

NOVO : real

inicio

escreva ("Informe o nome do funcionário: ")

leia (NOME)

escreva ("Informe a idade do funcionário: ")

leia (idade)

escreva ("Informe o salário atual: ")

leia (SAL)

NOVO <- SAL+10/100*SAL

escreva ("O novo salário é de: ", NOVO)

Fimalgoritmo



Atente-se para o fato de que em ambos os exemplos há uma padronização no uso dos comandos LEIA e ESCREVA, sendo que primeiro é apresentado o ESCREVA, informando ao usuário o tipo de dado a ser digitado e na sequência o comando que irá armazenar esses dados inseridos.



IV

Unidade IV - Estruturas Básicas de Controle



Objetivos da Unidade

- ✓ Conhecer as estruturas básicas de controle de um algoritmo;
- ✓ Compreender e aplicar estruturas condicionais;
- ✓ Compreender e aplicar estruturas de repetição;
- ✓ Compreender as principais regras para construção de algoritmos.



Plano de Estudos

- ✓ Ciclo 04
- ✓ Atividade Exercício em Grupo
- ✓ Título: Pensando Logicamente

Na unidade anterior, foram apresentados os comandos básicos de um algoritmo: declaração de variáveis, comandos de entrada e de saída e expressões.

Nesta unidade, serão estudadas as estruturas básicas de controle: sequência, estrutura condicional, estrutura de repetição e as principais regras para construção de algoritmos legíveis.

4.1 Sequência



Um algoritmo é formado por uma sequência de comandos. Uma sequência de comandos é formada por um grupo de comandos que são executados através da técnica top-down, ou seja, de cima para baixo, um comando após o outro.

Portanto, após a execução de um comando, ele não será mais executado, a não ser que esteja especificado dentro de uma estrutura de repetição, que veremos mais a frente nesta unidade. Observe o exemplo abaixo:



Algoritmo

"NomeAlgoritmo"

Comando_1;

Comando_2;

Comando_3;

Comando_4;

Comando_5;

Comando_n;

Fimalgoritmo

O Comando_1 será executado primeiro; após sua execução, teremos o Comando_2, e assim sucessivamente. Os comandos já executados não poderão ser executados novamente, a não ser que isso esteja especificado por um comando próprio, o de repetição.

4.2 Estrutura Condicional (Estrutura de Seleção)

Conceito

Também chamada de estrutura de seleção ou estrutura de decisão, a estrutura condicional é utilizada quando a execução de um comando (ou uma sequência de comandos) depende de um teste anterior (uma ou mais comparações).

Os comandos vistos até aqui não nos capacitaram a resolver problemas nos quais a tomada de decisão é fator importante. Imagine um problema que leia as notas de um aluno, determine a sua média e escreva se ele foi aprovado ou reprovado, considerando a média para aprovação maior ou igual a seis.

Faz-se necessária a introdução de um novo comando, que nos permitirá tomar uma atitude, dependendo de uma condição, isto é, precisamos das chamadas **estruturas condicionais**.



Estrutura condicional permite a escolha do comando ou grupo de comandos a ser executado quando determinadas condições são satisfeitas.

Pode-se apresentar de duas maneiras: estrutura condicional simples e estrutura condicional composta.

4.2.1 Estrutura Condicional Simples

A sintaxe do comando **se** para seleção simples é apresentada a seguir.

Forma Geral:

	<pre> se [condição] entao [comando ou] [bloco de comandos] fimse </pre>
---	--

onde:

- **se**, **entao** e **fimse** são palavras-chave;
- Condição é uma expressão lógica.

Observação: Atente-se para o uso do “fimse” sempre que utilizar um “se”. Sempre que você utilizar um “se” deverá, obrigatoriamente, ter um “então”.

Exemplos da estrutura “SE”:



<pre> se (A > B) entao escreva (A); fimse </pre>	<pre> se (X = Y E TESTE) entao Z <- X + Y; escreva (Z); fimse </pre>
--	---

Para fixar a estrutura condicional simples do “SE”, verifique os exemplos completos abaixo:



```
1 Algoritmo "Alg 12 – ExemploSelecaoSimples"
2   var A : inteiro
3 inicio
4   escreva ("Digite um número:")
5   leia (A);
6 se (A = 6) entao
7     escreva ("Condição Verdadeira!")
8     escreva ("O valor é", A)
9 fimse
10 finalgoritmo
```



O Algoritmo acima identifica se o número lido é 6. As instruções (linhas 6 e 7) dentro da estrutura se serão executadas dependendo do valor introduzido. Caso o valor lido em A seja diferente de 6, as instruções (linhas 6 e 7) dentro da estrutura se não serão executadas.

Considerando o exemplo anterior, verifique a seguir como estaria a saída na tela e a memória ao final da execução desse algoritmo. Considere ainda que o usuário digitou o valor 6 para variável A.

No vídeo será apresentado:

```
Digite um número: 6
Condição Verdadeira!
O valor é 6
```

Na memória será alocado o valor 6 para a variável A. Supondo que fosse digitado o valor 5, no vídeo seria apresentado:

Digite um número: 5

Na memória seria alocado o valor 5 para a variável A, porém nenhuma das mensagens anteriores seriam apresentadas, uma vez que a condição não foi verdadeira.



PARA PENSAR! Perceba que, ao longo dos exemplos que foram apresentados, algumas palavras apresentaram recuos à direita e outras mais ainda, você saberia dizer o motivo dessa formatação? Você saberia dizer qual o nome é dado para esse tipo de técnica em programação?

Para fixarmos a utilização da principal e mais utilizada estrutura das linguagens de programação e de qualquer programa de computador - a estrutura “SE” -, vamos analisar alguns exemplos.



Suponha que você precise desenvolver um algoritmo que leia dois números e os apresente em ordem crescente. Como poderíamos solucionar esse problema? Lembre-se que para sabermos se um número é maior que o outro é preciso compará-los. Essa comparação pode ser feita com os operadores, já vistos anteriormente, e utilizados em conjunto com a estrutura “SE”. Neste problema, precisaremos realizar três comparações: verificar se o primeiro número é maior que o segundo, se o segundo é maior que o primeiro e se eles são iguais.



Algoritmo “Alg 13 – LeDoisNumeros”

```
var A, B : inteiro;
inicio
leia (A, B)
se (B > A) entao
escreva (“B é maior que A:”, A, B)
fimse
se (A > B) entao
escreva (“A é maior que B:”, B, A)
fimse
se (A = B) entao
escreva (“A e B são iguais”)
fimse
finalgoritmo
```



Observação: Ao longo da vida de programador, torna-se perceptível que um mesmo problema pode ser resolvido de várias formas, porém haverá sempre a melhor forma de ser desenvolvido o algoritmo. No caso do exemplo acima, essa não é a melhor forma. Você sabe dizer o motivo? Discuta com os colegas e crie um algoritmo com a melhor solução!

No próximo exemplo, teremos que resolver uma equação do segundo grau. Na verdade, quem vai resolver para nós é o computador, pois iremos criar o programa que resolve uma equação do segundo grau.



Algoritmo “Alg 14 – EquacaoSegundoGrau”

```

var A, B, C, R1, R2 : inteiro
    DELTA : real
leia (A, B, C)
se (A <> 0) entao
    DELTA <- B^2-4*A*C
se (DELTA > 0) entao
    R1 <- -B-RAIZ(DELTA)/(2*A)
    R2 <- -B+RAIZ(DELTA)/(2*A)
    escreva (R1, R2)

fimse
se (DELTA = 0) entao
    R1 <- -B/(2*A)
    escreva (R1)

fimse
se (DELTA < 0) entao
    escreva (“Não tem Raiz Real”)

fimse
fimse
se (A = 0) entao
    escreva (“Não é uma equação do 2º
grau”)
fimse
finalgoritmo
  
```

Anteriormente, foi questionada a necessidade de se organizar o algoritmo com recuos à direita, alinhando o “SE” com “fimse”, e com um recuo maior os comandos que ali serão executados. O nome desse tipo de técnica é “IDENTAÇÃO” e pode ser considerada uma boa prática de programação, uma vez que programadores que não fazem uso da indentação acabam criando códigos ilegíveis. Veja o mesmo código acima sem indentação.



```

Algoritmo "Alg 15 – EquacaoSegundoGrauSemIdentacao"
var A, B, C, R1, R2 : inteiro
    DELTA : real
inicio
    leia (A, B, C)
    se (A <> 0) entao
        DELTA <- B^2-4*A*C
        se (DELTA > 0) entao
            R1 <- -B-RAIZ(DELTA)/(2*A)
            R2 <- -B+RAIZ(DELTA)/(2*A)
            escreva (R1, R2)
        fimse
        se (DELTA = 0) entao
            R1 <- -B/(2*A)
            escreva (R1)
        fimse
        se (DELTA < 0) entao
            escreva ("Não tem Raiz Real")
        fimse
    fimse
    se (A = 0) entao
        escreva ("Não é uma equação do 2º grau")
    fimse
finalgoritmo
    
```

Forma Incorreta!

Como você pôde perceber no algoritmo anterior, a falta de indentação acarreta em uma grande dificuldade no entendimento do código, dificultando futuras manutenções, até mesmo para o programador que desenvolveu o código.

Outro problema que pode ser verificado no código anterior é a grande utilização da estrutura condicional simples "SE e ENTÃO". Para solucionar ou minimizar a utilização desse tipo de estrutura em algoritmos, temos disponível a estrutura condicional composta.

4.3 Estrutura condicional composta

A estrutura condicional composta permite executar um comando (ou uma sequência de comandos) se a condição é verdadeira. E se a condição for falsa, se executa outro comando (ou outra sequência de comandos).

A sintaxe do comando se para seleção composta é apresentada a seguir:

Forma Geral:



```

se [condição]
    entao [comando ou]
        [bloco de comandos]
    senao [comando ou]
        [bloco de comandos]
fimse
    
```

onde:

- ✓ se, então, senão e fimse são palavras-chave;
- ✓ condição é uma expressão lógica.

Observação: cada **se** corresponde a um único **entao**, um único **senao** e um único **fimse**.

Agora que já temos conhecimento da estrutura condicional composta, podemos refazer os exemplos anteriores. Primeiramente, vamos refazer o exemplo que lê dois números e os apresenta em ordem decrescente.



Algoritmo "Alg 16 - LeDoisNumeros"

var A, B : inteiro

inicio

leia (A, B)

se (B > A) **entao**

escreva ("B é maior que A:", A, B)

fimse

se (A > B) **entao**

escreva ("A é maior que B:", B, " ", A)

senao

escreva ("A e B são iguais")

fimse

Fimalgoritmo

Agora vamos refazer o exemplo que calcula uma equação do segundo grau.



Algoritmo "Alg 19 - EquacaoSegundoGrau"

var A, B, C : inteiro

 DELTA, R1, R2 : real

inicio

leia (A, B, C)

se (A <> 0) **entao**

 DELTA <- $B^2 - 4 * A * C$

se (DELTA > 0) **entao**

 R1 <- $-B - \text{RAIZ}(\text{DELTA}) / (2 * A)$

 R2 <- $-B + \text{RAIZ}(\text{DELTA}) / (2 * A)$

escreva (R1, R2)

senao

se (DELTA = 0) **entao**

 R1 <- $-B / (2 * A)$

escreva (R1)

senao

escreva ("Não tem Raiz Real")

fimse

fimse

```

senao
    escreva ("Não é uma equação do 2º grau")
fimse
finalgoritmo

```

Percebe-se facilmente que, com a utilização da estrutura condicional composta, algoritmos com muitas condições ficam mais legíveis, o código fica menor e mais bem apresentável, ou seja, de fácil leitura e manutenção por parte dos programadores. A indentação é de extrema importância durante a codificação, é função do programador organizar o seu código segundo as melhores práticas de desenvolvimento.

4.4 Estruturas de repetição

A estrutura de repetição permite que uma sequência de comandos seja executada repetidamente até que uma determinada condição de interrupção seja satisfeita.

A condição de interrupção que deve ser satisfeita é representada por uma expressão lógica. Todos os algoritmos apresentados até agora permitem a execução de um comando ou bloco de comandos uma única vez. Analise o exemplo abaixo: Calcular e escrever a média aritmética das 4 notas bimestrais de um aluno.



```

Algoritmo "Alg 20 - CalculaMedia"
var N1, N2, N3, N4 : inteiro
    MEDIA : real
    NOMEALUNO : caractere
inicio
    leia (NOMEALUNO, N1, N2, N3, N4)
    MEDIA <- (N1+N2+N3+N4)/4
    escreva (NOMEALUNO, MEDIA)
finalgoritmo

```

Para calcular e escrever a média das notas de dois alunos sem abandonar e retornar ao algoritmo, você teria as seguintes opções:

Opção I: Definir dois jogos de variáveis, um para cada aluno (INCORRETO).



Algoritmo "Alg 21 - CalculaMedia"

var A1, A2, A3, A4 : inteiro

B1, B2, B3, B4 : inteiro

MEDIAA, MEDIAB : real

NOMEALUNOA : caractere

NOMEALUNOB : caractere

inicio

leia (NOMEALUNOA, A1, A2, A3, A4)

leia (NOMEALUNOB, B1, B2, B3, B4)

MEDIAA <- (A1 + A2 + A3 + A4)/4

MEDIAB <- (B1 + B2 + B3 + B4)/4

escreva (NOMEALUNOA, MEDIAA)

escreva (NOMEALUNOB, MEDIAB)

fimalgoritmo

Perceba o quão trabalhoso seria se tivéssemos 20 ou 30 alunos para realizar esse procedimento, totalmente inviável. Outra opção seria utilizarmos o mesmo conjunto de variáveis, o que também está **INCORRETO**.



Algoritmo "Alg 22 - CalculaMedia"

var N1, N2, N3, N4 : inteiro

MEDIA : real

NOMEALUNO : caractere

inicio

{Primeiro aluno}

leia (NOMEALUNO, N1, N2, N3, N4)

MEDIA <- (N1+N2+N3+N4)/4

escreva (NOMEALUNO, MEDIA)

```
{Segundo aluno}
  leia (NOMEALUNO, N1, N2, N3, N4)
  MEDIA <- (N1+N2+N3+N4)/4
  escreva (NOMEALUNO, MEDIA)
finalgoritmo
```

Com essa nova versão, temos o mesmo número de variáveis usadas no algoritmo que calcula a média de um único aluno. Observe que os dois blocos assinalados repetem a sequência de comandos necessários para calcular a média de cada aluno, o que nos leva a concluir que se tivéssemos 1000 alunos, esses blocos deveriam se repetir 1000 vezes. Então, repetiremos isso usando o seguinte algoritmo:



Algoritmo "Alg 23 - EstruturaRepeticao"

```
var N1, N2, N3, N4 : inteiro
    MEDIA : real
    NOMEALUNO : caractere
inicio
repita
  leia (NOMEALUNO, N1, N2, N3, N4)
  MEDIA <- (N1+N2+N3+N4)/4
  escreval (NOMEALUNO, MEDIA)
fimrepita
finalgoritmo
```



OBSERVAÇÃO: Tendo como base o algoritmo anterior, eu lhe pergunto: para quantos alunos esse algoritmo irá executar? A resposta é: para o infinito! Ou seja, teríamos um loop infinito. Para que esse tipo de problema ocorra, uma estrutura de repetição deverá ter sempre uma condição de parada.

A Estrutura de repetição permite que uma sequência de comandos seja executada repetidamente até que uma condição determinada seja satisfeita, o que fará a interrupção do processo.

Essa estrutura poderá se apresentar de 3 maneiras:

- ✓ Comando Repita.
- ✓ Comando Enquanto.
- ✓ Comando Para.

A utilização de cada uma delas depende do problema a ser solucionado.

4.4.1 Comando Repita

O comando Repita pode ser construído de três formas:

- ✓ Interrupção no Início
- ✓ Interrupção no meio
- ✓ Interrupção no final

A utilização de cada uma delas depende do problema a ser solucionado.

Interrupção no Início - Forma Geral:



```
repita
    se [condição]
        entao interrompa;
    fimse
    [comando ou]
    [bloco de comandos]
fimrepita
```

onde:

repita, fimrepita e interrompa são palavras-chave.



OBSERVAÇÃO: Atente-se para o fato de que, caso a estrutura condicional simples “SE” for verdadeira, a repetição já é interrompida antes de executar qualquer comando.

Conceito

[Condição] é uma expressão lógica que, se verdadeira, determina a interrupção do processo de repetição. No algoritmo anterior, queríamos limitar a interrupção quando o número de alunos fosse igual a 1000. Isso nos leva a concluir que temos que contar os alunos para sabermos se já completamos os 1000.

Precisamos, então, de uma variável contadora que chamaremos de, por exemplo, CONTADOR. Como, obviamente, vamos começar do primeiro aluno, iniciaremos o CONTADOR com o valor 1. Portanto, o algoritmo que calcula a média dos alunos ficaria da seguinte forma:

Exemplo - Estrutura de Repetição com interrupção no início



```
Algoritmo "Alg 24 - ExemploEstruturaRepeticao"  
var N1, N2, N3, N4, CONTADOR : inteiro  
    MEDIA : real  
    NOMEALUNO : caractere  
inicio  
{Inicializando o valor de contador}  
CONTADOR <- 1  
repita  
    se (CONTADOR > 1000) entao  
        interrompa  
    fimse  
    leia (NOMEALUNO, N1, N2, N3, N4)  
    MEDIA <- (N1+N2+N3+N4)/4  
    escreva (NOMEALUNO, MEDIA)  
{Incrementar o contador}  
    CONTADOR <- CONTADOR+1  
fimrepita  
finalgoritmo
```

Assim, teremos uma estrutura de repetição que será interrompida quando calcular a média de 1000 alunos. Observe que, se alterarmos a condição de parada, podemos resolver o problema para qualquer número de alunos.

Vamos a outro exemplo em que é necessário que sejam escritos os 100 primeiros números naturais iniciando em zero.



```

Algoritmo "Alg 25 - Escreve 100Numeros"
var NUMERO : inteiro
inicio
    NUMERO <- 0;
    repita
        se (NUMERO > 100) entao
            interrompa
        fimse
    escreva (NUMERO)
    NUMERO <- NUMERO+1
    fimrepita
finalgoritmo
    
```



Observação: esse algoritmo não possui comando de entrada porque os números a serem escritos não são quaisquer; eles variam de 0 a 100 de 1 em 1. Quando ao fim da execução do algoritmo, o último NUMERO armazenado na memória será 101, que não será escrito, pois quando isso acontecer a estrutura será interrompida, desviando o fluxo para o primeiro comando após o **fimrepita** e, nesse caso, para o **finalgoritmo**.

Vamos analisar mais um exemplo. Ler N números inteiros e apresentar na tela apenas os números que são pares.



Antes de iniciar o algoritmo, você deve estudar o problema: o usuário deverá fornecer ao algoritmo a quantidade de números que se deseja ler. Em seguida, fornecer cada número. Para interrompermos a estrutura de repetição, devemos ir contando os números, o que requer uma variável contadora que deverá ser iniciada (ou "inicializada") com 1, indicando o primeiro número a ser fornecido.

A quantidade de números deverá ser fornecida uma única vez, o que indica que este comando leia deve ficar fora da estrutura de repetição, enquanto que a leitura de cada número se repetirá, implicando que este comando leia deverá constar dentro da estrutura de repetição e, após cada uma destas leituras, o contador deverá ser acrescido de um. Sabemos também que uma maneira de se determinar se o número é par consiste em fazer uma divisão inteira deste por 2; se o resto dessa divisão for igual a zero, a paridade estará garantida. Para isso, podemos fazer uso da função MOD (Resto). Basta, agora, fazermos o algoritmo:



Algoritmo "Alg 26 - ApresentarPares"
var NUMERO, CONTADOR, QUANT_NUM : inteiro
inicio
 CONTADOR <- 1
leia (QUANT_NUM) {quantidade de números}
repita
 se (CONTADOR > QUANT_NUM) **entao**
 interrompa
 fimse
escreva ("Digite um novo numero: ")
leia (NUMERO)
 se (NUMERO) MOD 2 = 0 **entao** {Função MOD calcula

```

o RESTO de uma divisão}
  escreva (NUMERO)
senao
  escreva ("O numero é ímpar!")
fimse
CONTADOR <- CONTADOR+1
fimrepita
finalgoritmo

```

Antes de passarmos para o próximo exemplo, vamos introduzir a noção de somadores. Imagine que você (não o algoritmo) deseja obter a soma de um conjunto de números, por exemplo:

1	2	4	5	1	2	3
---	---	---	---	---	---	---

Se lhe perguntassem qual a soma, você responderia imediatamente, sem dificuldades: 18. E, se lhe perguntassem como você chegou a esse resultado, temos a certeza que sua resposta seria: “somando, claro!!!”. Mas como? Talvez agora você tivesse dificuldade em responder com clareza. Você não efetuou a soma de uma só vez, mas sim dois a dois, somando sempre o resultado com o próximo número.

Numericamente, você faz:

1ª soma = 3 -> SOMA

2ª soma = 3 + 4 = 7 -> SOMA + 4

3ª soma = 7 + 5 = 12 -> SOMA + 5

4ª soma = 12 + 1 = 13 -> SOMA + 12

5ª soma = 13 + 2 = 15 -> SOMA + 13

6ª soma = 15 + 3 = 18 -> SOMA + 15

Observe que, com exceção da 1ª soma, todas as outras podem ser formuladas como $SOMA \leftarrow SOMA + NUMERO$, o que será válido para a 1ª vez,

se a nossa soma inicial for colocada como zero. Aí, a 1ª SOMA = 3 <- 0 + NUMERO e a regra estará definida.

Tendo como base essas informações, vamos criar um algoritmo que lê um conjunto de números N e escreva a soma dos positivos.



```

Algoritmo "Alg 27 - SomaPositivos"
var NUMERO, CONTADOR, QUANT_NUM, SOMA : inteiro
inicio
    CONTADOR <- 1
    SOMA <- 0
    escreva ("Digite quantidade de repetições: ")
    leia (QUANT_NUM) {quantidade de números}
    repita
        se (CONTADOR > QUANT_NUM) entao
            interrompa
        fimse
        escreva ("Digite um novo numero: ")
        leia (NUMERO)
        se (NUMERO > 0) entao
            SOMA <- SOMA+NUMERO
        fimse
        CONTADOR <- CONTADOR+1
    fimrepita
    escreva (SOMA)
finalgoritmo
    
```

Observe que o comando escreva SOMA ficou fora da estrutura de repetição, porque o que nos interessa é o resultado final da soma e não os subtotais.

Estrutura de repetição com Interrupção no Meio: Em alguns problemas, faz-se necessário o uso de algum comando dentro da estrutura de repetição, antes da condição de parada.

Por exemplo, se quiséssemos ler as notas e o número de matrícula de vários alunos sem nos preocuparmos inicialmente com quantos alunos devemos introduzir uma condição de parada tipo: para se o número de matrícula for igual a zero. Para isso, antes de decidirmos se o processo será ou não interrompido, temos que ler esse número de matrícula. Precisamos, então, de uma estrutura em que a interrupção se processe no meio do bloco.

Forma Geral:



```

repita
    [comando ou]
    [bloco de comandos]
se [condição]
        entao interrompa;
fimse
    [comando ou]
    [bloco de comandos]
fimrepita
    
```

A escolha entre a estrutura com interrupção no início e interrupção no meio vai depender do problema a ser resolvido e, algumas vezes, da preferência da pessoa que está escrevendo o algoritmo. Isso porque um só problema pode fazer uso tanto de um tipo de interrupção como de outro.



Leia o número de matrícula e as 4 notas bimestrais dos alunos de uma determinada disciplina. Calcule e escreva a sua média, acompanhada do número de matrícula e a expressão “aprovado” ou “reprovado”, considerando a média para aprovação maior ou igual a 6. Como o problema não determina o número total de alunos, notamos que se faz necessário o uso da estrutura de repetição com interrupção no meio, pois a condição de parada depende do número de matrícula cuja leitura constará da estrutura.



```

Algoritmo "Alg28 - CalculaMedia"
var MATRICULA, N1, N2, N3, N4 : inteiro
    MEDIA : real
inicio
repita
escreva ("Digite a matricula:")
leia (MATRICULA)
se (MATRICULA = 0) entao
    interrompa
fimse
escreva ("Digite a primeira nota:")
leia (N1)
escreva ("Digite a segunda nota:")
leia (N2)
escreva ("Digite a terceira nota:")
leia (N3)
escreva ("Digite a quarta nota:")
leia (N4)
MEDIA <- (N1 + N2 + N3 + N4) / 4
se (MEDIA >= 6) entao
    escreval ("Matricula:", MATRICULA, " Media: ", MEDIA,
"Situação: Aprovado")
senao
    escreval ("Matricula:", MATRICULA, " Media: ", MEDIA,
"Situação:

```

Observe que as variáveis "MATRICULA e MEDIA" foram escritas no **entao** e no **senao**.

Isso poderia ser simplificado? Como?

Interrupção no Final: Podemos também fazer uso da estrutura de repetição com interrupção no final. A escolha de cada estrutura depende do problema a ser resolvido e da lógica individual.

Forma Geral:



```
repita
    [comando ou]
    [bloco de comandos]
    se [condição]
        entao interrompa;
    fimse
fimrepita
```

Vamos verificar como ficaria um exemplo já apresentado anteriormente.
Escrever os números naturais de 0 a 100.



```
Algoritmo "Alg 29 - CalculaNaturais"
var NUMERO : inteiro
inicio
    NUMERO <- 0
    repita
        escreval (NUMERO)
        NUMERO <- NUMERO+1
        se (NUMERO > 100) entao
            interrompa
        fimse
    fimrepita
finalgoritmo
```

4.4.2. A estrutura de repetição Enquanto

O comando Enquanto é equivalente ao comando Repita com interrupção no início.

Forma Geral:



```
enquanto [condição] faça  
    [comando ou]  
    [bloco de comandos]  
fimenquanto
```

OBSERVAÇÃO: Enquanto o valor da [condição] for verdadeiro, as ações dos comandos são executadas. Quando for falso, o comando é abandonado. Se já da primeira vez o resultado é falso, os comandos não são executados nenhuma vez.

Vamos escrever o algoritmo anterior, que apresenta os números naturais de 0 a 100, utilizando o comando enquanto.



```
Algoritmo "Alg30 - EstruturaEnquanto"  
var NUMERO : inteiro  
inicio  
    NUMERO <- 0  
    enquanto (NUMERO <= 100) faca  
        escreval (NUMERO)  
        NUMERO <- NUMERO+1  
    fimenquanto  
fimalgoritmo
```

Atente-se para a estrutura do enquanto, que utiliza não só o **enquanto**, mas também o **faça**. Caso apenas um deles seja utilizado, o algoritmo apresentará erro.

4.4.3 Estrutura de repetição para

O comando PARA é, na verdade, o comando enquanto utilizando uma variável de controle, escrito numa notação compactada. Nesse caso, existirá sempre uma inicialização da variável de controle, um teste para verificar se a variável atingiu o limite e um acréscimo na variável de controle.

Forma Geral:



```
para [variável] de [Num1] até [Num2] passo
[Num3] faça
    [comando ou]
    [bloco de comandos]
Fimpara
```

Aparentemente, a estrutura PARA parece complicada, mas não é, pelo contrário, é uma estrutura que simplifica a utilização de repetições.

Onde:

[variável] - Variável de controle tipo numérico (contador);

[Num1] - Valor numérico que representa o estado inicial do contador;

[Num2] - Valor numérico que representa o estado final do contador;

[Num3] - Valor numérico que representa o incremento do contador.

Vamos ao exemplo de escrever os 100 números naturais, agora utilizando a estrutura PARA.



```
Algoritmo "Alg31 - CalculaComPara"
var NUMERO : inteiro
inicio
para NUMERO de 0 ate 100 passo 1 faça
    escreval (NUMERO)
fimpara
finalgoritmo
```

Observação: Caso fôssemos descrever a estrutura PARA anterior, poderíamos fazê-lo da seguinte forma: Para a variável NUMERO iniciando em 0 e indo até 100, sendo incrementada de 1 em 1, escreva esta variável.

4.5 Regras para a construção de algoritmos legíveis



- ✓ Procure incorporar comentários no algoritmo, pelo menos para descrever o significado das variáveis utilizadas.

Exemplo: {Cálculo da média}

- ✓ Escolha nomes de variáveis que sejam significativos, isto é, que traduzam o tipo de informação a ser armazenada na variável. Exemplos: NOTA, MATRÍCULA.
- ✓ Procure alinhar os comandos de acordo com o nível a que pertençam, isto é, destaque a estrutura na qual estão contidos.
- ✓ Tenha em mente que seus algoritmos deverão ser lidos e entendidos por outras pessoas (e por você mesmo) de tal forma que possam ser corrigidos, receber manutenção e serem modificados.
- ✓ Escreva comentários no momento em que estiver escrevendo o algoritmo. Um algoritmo não documentado é um dos piores erros que um programador pode cometer.



- ✓ Use comentários no início para explicar o que o algoritmo faz. Alguns comentários seriam:
 - Uma descrição do que faz o algoritmo;
 - Como utilizá-lo;
 - Explicação do significado das variáveis mais importantes;
 - Estruturas de dados utilizadas;
 - Os nomes de quaisquer métodos especiais utilizados;
 - Autor;
 - Data da escrita.

- ✓ Utilize espaços em branco para melhorar a legibilidade. Espaços em branco, inclusive linhas em branco são valiosíssimos para melhorar a aparência de um algoritmo.
- ✓ Um comando por linha é suficiente
 - O algoritmo fica mais legível;
 - O algoritmo fica mais fácil de ser depurado.
- ✓ Utilize parênteses para aumentar a legibilidade e prevenir-se contra erros.
- ✓ Toda vez que uma modificação for feita no algoritmo, os comentários devem ser alterados.



Dicas ao iniciar a resolução de um algoritmo:

Passo 1 – Leia cuidadosamente a especificação do problema até o final;

Passo 2 – Caso não tenha entendido o problema, pergunte;

Passo 3 – levantar e analisar todas as saídas exigidas na especificação do problema;

Passo 4 – levantar e analisar todas as entradas citadas na especificação do problema (leituras);

Passo 5 – Verificar se é necessário gerar valores internamente ao algoritmo e levantar as variáveis necessárias e os valores iniciais de cada uma;

Passo 6 – Levantar e analisar todas as transformações necessárias para, dadas as entradas e valores gerados internamente, produzir as saídas especificadas;

Passo 7 – Testar cada passo do algoritmo;

Passo 8 – Avaliação geral, elaborando o algoritmo e revendo comentários.

4.5.1 Tipos abstratos de dados

Neste tópico, serão apresentadas as variáveis compostas homogêneas que correspondem a posições de memória, identificadas por um mesmo nome, individualizadas por índices e cujo conteúdo é de mesmo tipo. As variáveis compostas homogêneas podem ser: vetor ou matriz.

4.5.2 Vetor (Matriz Unidimensional)

Conceito

Um vetor é uma estrutura de dados que contém elementos de mesmo tipo, que podem ser referenciados como um todo. Ao declararmos um vetor, estamos reservando na memória principal do computador uma série de células para uso da variável daquele tipo.

O nome do vetor aponta para a base das células e o seu início dá a posição relativa do elemento referenciado ao primeiro (base).

Nem sempre os tipos básicos (numérico, literal e lógico) são suficientes para exprimir estruturas de dados em algoritmos.



Por exemplo, consideremos um problema em que um professor com 5 alunos deseja imprimir a nota e a média de seus alunos. Nesse caso, seria necessário considerar cinco variáveis reais para contar as notas dos cinco alunos. Imagine que o número de alunos da turma seja 80. Só a declaração dessas variáveis tornaria impraticável a redação do algoritmo. Daí a necessidade de novos tipos serem criados.

Os vetores podem ser unidimensionais ou multidimensionais (matrizes). Um vetor unidimensional, como uma lista de notas dos 50 alunos de uma turma, tem apenas um índice. Se existirem, porém, várias turmas, poderemos utilizar um vetor com dois índices (o número da turma e o número do aluno da turma).

Forma Geral:



declare NOME_VARIAVEL[LI : LF] tipo;

Onde:

- NOME_VARIAVEL – Nome da variável heterogênea.
- [LI : LF] – LI é o limite inferior e LF o limite superior (final).
- Tipo – Pode ser numérico, lógico ou literal.

O número de elementos de um vetor é dado por: $LF - LI + 1$. Isso significa que as posições do vetor são identificadas a partir de LI, com incrementos unitários, até LF:

LI	LI+1		LF
----	------	------	----

Declaração de um vetor:



declare NOTAS[1:80] numérico;

Esse vetor contém oitenta elementos do tipo numérico.



Qual o número de elementos é o que representa a especificação e declaração abaixo?

declare NOME[1:5] literal;

Solução:

O vetor tem $5 - 1 + 1 = 5$ (cinco) elementos e pode ser representado como na figura abaixo.

1	2	3	4	5

Cada elemento de um vetor é tratado como se fosse uma variável simples. Para referência a um elemento do vetor, utiliza-se o nome do vetor e a identificação do elemento (índice) entre colchetes. Por exemplo, se desejarmos atribuir o valor "Teste" ao elemento identificado pelo índice 3 do vetor anterior, teremos:

Nome [3] <- "Teste";

Essa atribuição irá produzir:

1	2	3	4	5
		Teste		

Caso seja feita a seguinte atribuição: Nome [6] <- "Rodrigo";

Um erro de sintaxe será retomado, pois o vetor possui apenas cinco posições, não existindo a sexta. Vamos avaliar um exemplo. Criar um algoritmo que escreva a média de notas de uma sala de aula com 50 alunos.



Algoritmo "Alg 32 - UsandoVetorI"

var

SOMA, CONTADOR : inteiro

MEDIA : real

X : vetor [1..80] de inteiro

inicio

SOMA <- 0

para CONTADOR de 1 **ate** 80 **PASSO** 1 **faca**

escreva("Digite o valor da posição ",
CONTADOR, ";")

leia (X[CONTADOR])

 SOMA <- SOMA + X[CONTADOR]

fimpara;

MEDIA <- SOMA/80

escreva (MEDIA)

fimalgoritmo

4.5.3 Matriz (Matriz Multidimensional)

A matriz é um conjunto de dados referenciados por um mesmo nome e que necessita de mais de um índice para ter seus elementos individualizados.

Forma Geral:



declare NOME_VARIAVEL[L11 : LF1, L12 : LF2] tipo;

Onde:

- NOME_VARIAVEL – Nome da variável heterogênea.
- [LI1 : LF1, LI2 : LF2] – LI1 é o limite inferior e LF1 o limite superior (final), LI2 é o limite inferior e LF2 o limite superior (final).
- Tipo – Pode ser numérico, lógico ou literal

Essa especificação corresponde apenas à criação do modelo e, para efetivar a estrutura de dados dentro do algoritmo, é necessária uma declaração dando um nome à variável que segue o modelo.

Declaração de uma matriz:



declare MATRIZ[1:3][1:4] numérico;

O exemplo acima declara uma matriz com as seguintes dimensões:

	1	2	3	4
1				
2				
3				

Para referenciar um elemento da matriz, são necessários tantos índices quantas são as dimensões da matriz. Numa matriz bidimensional (duas dimensões), o primeiro índice indica a linha; e o segundo, a coluna. Para atribuir o valor 10 à posição 2,3 da matriz anterior, devemos realizar a seguinte declaração.

MATRIZ [2,3] <- 10

Para verificarmos a utilização de uma matriz, vamos criar um algoritmo que atribua o valor zero a todas as posições de uma matriz 4 x 4.



```

Algoritmo "Alg 33 - ExemploMatriz I"
var
    I, J : inteiro
    MATRIZ: vetor [1..4, 1..4] de inteiro
inicio
    para I de 1 ate 4 passo 1 faca
        para J de 1 ate 4 passo 1 faca
            MATRIZ[I, J] <- 0
        fimpara;
    fimpara;
fim algoritmo
    
```

O exemplo anterior demonstra o preenchimento de uma matriz 4 x 4 com o valor 0, atente-se para a forma como pode ser feita a varredura de uma matriz. O primeiro PARA indica a linha; e o segundo, as colunas. Assim, uma matriz é preenchida com a seguinte sequência: [Linha 1, coluna 1], [linha 1, coluna 2], [linha 1, coluna 3], [linha 1, coluna 4], depois a sequência continua com a [linha 2, coluna 1], etc.



Unidade V - Introdução à Linguagem de Programação C



Objetivos da Unidade

- ✓ Conhecer a sintaxe básica da linguagem C;
- ✓ Declarar e manipular operadores, expressões e variáveis;
- ✓ Criar programas simples na linguagem de programação C.



Plano de Estudos

- ✓ Ciclo 05
- ✓ Atividade Exercício
- ✓ Título: Criando Algoritmos com Estruturas Avançadas

Nesta unidade vamos conhecer um pouco sobre uma das principais linguagens de programação, a linguagem C. Pode-se dizer que esta é a mais difundida e utilizada em todo o mundo, principalmente em ambiente acadêmico, não por ser uma linguagem criada na década de 70, mas pelas suas características que facilitam o desenvolvimento de programas de computador robustos e que são capazes de solucionar problemas dos mais diferentes tipos.

Ressalta-se que a linguagem C não é uma linguagem para o desenvolvimento de sistemas comerciais, sendo utilizada basicamente em problemas matemáticos e de manipulação de dados em memória principal.

5.1 O básico da linguagem C

Conceito

A linguagem C foi criada na década de 70 por Dennis Ritchie. O C é uma linguagem de programação genérica que é utilizada para a criação de programas diversos como processadores de texto, planilhas eletrônicas, sistemas operacionais, programas de comunicação, automação industrial, gerenciadores de banco de dados, programas para problemas da engenharia, física, matemática, entre outras áreas. Diversos softwares que utilizamos no dia a dia são desenvolvidos em C ou C++, que é uma versão orientada a objetos do C.

Há variações da linguagem C, porém a mais difundida é o padrão ANSI C, padronizado pela ANSI.

Estudar uma linguagem de programação nada mais é do que conhecer as suas sintaxes, funções e formas de estruturar a lógica do problema. Por isso, no início deste guia focamos em algoritmo, pois um programa em uma linguagem de programação é antes, ou pelo menos deve ser, um algoritmo em um pseudocódigo ou fluxograma.

5.2 A linguagem C é “Case Sensitive”

Um termo muito conhecido no ambiente de programação é “*Case sensitive*”, ou traduzindo, seria “sensível à caixa”, em que caixa quer dizer letra maiúscula ou minúscula. Ao declaramos uma variável precisamos identificá-la com um nome, esse nome pode ser: Aluno, ALUNO, aluno ou AlUnO. No C, todas essas formas são diferentes, ou seja, são consideradas variáveis diferentes, o que acaba confundindo o programador, caso ele não adote uma convenção logo no início do seu programa.



Recomenda-se que as variáveis sejam todas declaradas em caracteres minúsculos, ou que apenas a primeira letra seja maiúscula, de forma alguma misture as formas de declaração. Siga um padrão!

Observação: Existem linguagens de programação que não são “case sensitive”, ou seja, elas não diferenciam letras maiúsculas de minúsculas.

5.3 Configurando o ambiente de programação

Para que você crie os primeiros programas na linguagem C é necessário que seja instalado no seu computador um compilador ou uma IDE com compilador(es) embutido(s), existem vários para as mais diferentes plataformas operacionais, a seguir apresento os principais:

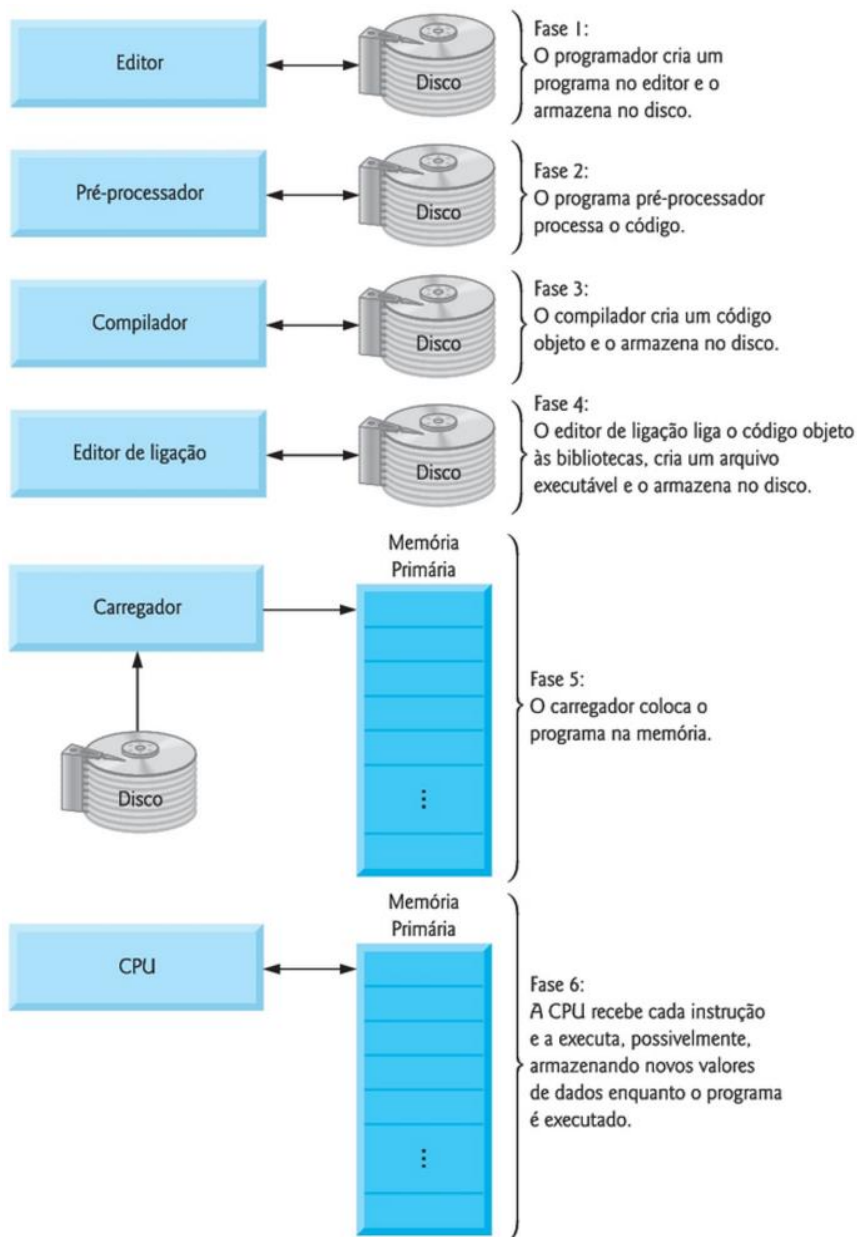
- **Code blocks:** É uma IDE gratuita para a criação de programas em C, C++ e FORTRAN. Disponível em: <<http://www.codeblocks.org/>>.
- **Dev-C++:** É uma outra IDE para o desenvolvimento de programas em C e C++, utiliza o GCC internamente como compilador. Disponível em: <<http://www.bloodshed.net/devcpp.html>>.

- **GNU GCC:** É um compilador nativo do Linux para a linguagem C. Disponível em: <<http://gcc.gnu.org/releases.html>>.

Existem outros, porém não é necessário citá-los aqui. Você pode utilizar aquele que melhor se adeque às suas necessidades, ou que tenha compatibilidade com o seu hardware. Sugiro que instale o Dev-C++.

A seguir é apresentado um esquema gráfico da forma como é desenvolvido um programa em C.

Figura 01: Ambiente típico de desenvolvimento em C



Fonte: Deitel & Deitel (2011), p. 10.

5.4 Tipos de dados, expressões e operadores em C

Todas as variáveis têm um tipo. O tipo define o espaço que a variável ocupará e os valores que ela poderá armazenar (DEITEL e DEITEL, 2011).

Alguns tipos de dados em C:

Tipo	Tamanho	Valores válidos
char	1 byte	Caracteres do código ASCII
int	4 bytes	(Somente inteiros)
float	4 bytes	-3.4×10^{38} a $+3.4 \times 10^{38}$ com até 6 dígitos de precisão
double	8 bytes	-1.7×10^{308} a $+1.7 \times 10^{308}$ com até 10 dígitos de precisão

Declaração de variáveis

Um tipo (por exemplo: *float*, *int*, *char*) seguido de uma lista de variáveis (de 1 a n).

	<pre>float valoremreais, taxaconvdolar; int valor1, valor2, valor3;</pre>
---	---

Constantes

São identificadores que não podem ter seus valores alterados durante a execução do programa. Como criar: com o comando `#define`, em geral colocado no início do programa.

	<pre>#define VALOR_DO_PI 3.1415 #define MAX_ALUNOS 20</pre>
---	---

Os nomes das constantes, como regra geral, são declarados em maiúsculas e **nada deve ser colocado entre o nome da constante e o seu valor.**

Sequência de caracteres entre aspas duplas.

	<pre>"isto é um string!"</pre>
---	--------------------------------

Expressões

Conjuntos de operandos (números, constantes, variáveis, etc.) agrupados por operadores, constituindo formas algébricas que representam um valor.

Operadores Aritméticos

Operador	Operação
+	Adição
*	Multiplicação
/	Divisão (inteira ou real)
%	Resto da divisão inteira
-	Subtração
--	Decremento
++	Incremento

Operadores Relacionais e Lógicos

Operador	Operação
>	Maior que
>=	Maior ou igual que
<	Menor que
<=	Menor ou igual que
==	Igual a
!=	Diferente de
&&	E
	OU
!	NÃO

As expressões devem ser linearizadas e todos os operadores devem ser explicitados.

Ex.: seja a expressão



$$S_n = (a_1 + a_n) \frac{n}{2}$$

Sua escrita em C:

```
soma_prog = (valor_inicial + valor_final) * (num_elem / 2);
```

Precedência dos operadores

A precedência dos operadores aritméticos segue conforme a Matemática tradicional. Para alterar a precedência de operadores usa-se tão somente parênteses.

Operador (comando) de atribuição: =

Atribui o resultado da expressão à direita do sinal de igual (=) à variável que está à esquerda. O valor anterior da variável é perdido.

Forma geral:

```
variavel = expressao;
```



```
valor1 = 4;  
increm = x + 2;  
valor2 = 2.5;  
semi_per = (lado1 + lado2+ lado3) / 2;
```

ERRO COMUM EM PROGRAMAÇÃO 01:



Erros de divisão por zero acontecem no momento em que o programa está sendo executado, por isso são chamados erros de “runtime”. Dividir por zero é um erro fatal e geralmente causa a finalização imediata do programa. Erros não fatais não causam a parada imediata da execução, porém provocam resultados incorretos.

5.5 Instrução de entrada scanf

Conceito

O comando scanf possibilita a leitura de dados via teclado. Converte os valores lidos em caracteres ASCII, para a representação interna, armazenando-os em variáveis.

Forma geral:

`scanf ("string de controle" , lista de argumentos);`

onde:

- String de controle: conterá especificações de formato, as quais indicam o tipo de valor a ser lido.

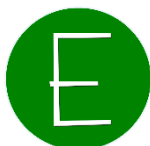
Em `scanf ("%d", &valor);` o `%d` é uma especificação de formato que corresponde a um valor inteiro.

- Lista de argumentos: Para cada especificação de formato, deverá existir na lista de argumentos a indicação de uma variável, que no exemplo anterior é `&valor`.



Nomes de variáveis no `scanf` devem ser precedidos por um e comercial (&).

Outros exemplos:



Para a leitura de um inteiro:

`scanf ("%d", &valorInteiro);`

Para a leitura de um real:

```
scanf("%f", &valorReal);
```

Para a leitura de um caractere:

```
scanf("%c", &soumchar);
```

Algumas especificações de formato:

Código	Formato
%c	Um caractere (char)
%d	Um número inteiro decimal (int)
%i	O mesmo que %d
%f	Ponto flutuante decimal

5.6 Instrução de saída printf

Conceito

O comando Printf possibilita a apresentação de valores constantes ou valores de expressões na tela.

Forma geral:

```
printf ("string de controle" , expressões);
```

onde:

- **String de controle:** Conterá especificações de formato e literais. As especificações de formato indicam o tipo de valor a ser apresentado. Em printf ("Numero de parcelas %d", numparc) o %d é uma especificação de formato que corresponde a um valor inteiro.
- **Expressões:** Para cada especificação de formato no interior da string de controle, deverá existir uma expressão após a vírgula que separa a string de controle das expressões. Mas se não houver na string de controle qualquer especificação de formato, também não haverá a indicação de qualquer expressão logo a seguir.



1) printf com string de controle sem especificação de formato:

```
printf("Mensagem só com literais, nenhuma expressão necessária!");
```

2) printf com três especificações de formato no string de controle, exigindo três expressões após a vírgula:

```
printf("%f reais = %f dolares, taxa = %f", reais, reais/taxa, taxa);
```



Observação: um `\n` dentro do string de controle provoca uma quebra de linha na tela.

Comentários na linguagem C

São quaisquer textos precedidos por `//` (do `//` até o final da linha), ou trechos entre `/*` e `*/`, que podem, nesse caso, compreender várias linhas.

ERRO COMUM EM PROGRAMAÇÃO 02:



Iniciar um comentário com os caracteres `/*` ou encerrá-lo com `*/`.

Vamos analisar um programa completo em C:

```
/* Programa 01 - Elementos básicos de um programa em C, somente o
essencial */
```

```
#include <stdlib.h>    // Arquivo de cabeçalho (header) –
                      Bibliotecas de funções

#include <stdio.h>     // Arquivo de cabeçalho (header)

#define MAXIMO 30     // Declaração de constante
```

```

valor                                     //Não se coloca ponto e vírgula após o

int main ( )                             // aqui inicia o programa principal – função
MAIN
{
    // comandos
    system("PAUSE");                      /* o pressionar de uma tecla fará retornar para
a tela de edição. */

    // só funciona se o include de stdlib estiver
    presente.

    return 0;                             //indica que o programa terminou com
    sucesso.
}

```

DICA DE PROGRAMAÇÃO 01:



Escreva os seus programas sempre de maneira simples e direta. Esta forma de programação é chamada de KIS (Keep it Simple – “Mantenha a simplicidade”). Não queira reinventar a roda, seja direto!

5.7 Uso de funções pré-definidas

Existem funções já prontas que podem ser usadas.



Por exemplo, para extrair uma raiz quadrada ou elevar um valor a uma certa potência. A função `sqrt` extrai a raiz quadrada e a função `pow` eleva um valor a uma certa potência.

Para usar qualquer uma dessas duas funções, no início do programa deve ser colocado um `# include <math.h>`.

A função **sqrt** exige um só parâmetro, o valor do qual se quer extrair a raiz quadrada.



```
raiz = sqrt(valor);
```

Os parâmetros da função **pow** são dois: o valor que deve ser elevado a uma certa potência e a potência a qual quer se elevar o valor.



```
val_ao_cubo = pow (valor, 3);
```

Ao usarmos funções pré-definidas, como **sqrt** e **pow**, os parâmetros também podem ser expressões. Nesse caso, a expressão será avaliada e o valor resultante usado como parâmetro da função.



```
det = sqrt(b*b - 4 * a * c);
```

Vamos analisar alguns exemplos simples. O primeiro irá apresentar na tela o nome “Bacharelado em Sistemas de Informação – UNIS/MG”.

```
/*Programa 02 - Apresenta uma string na tela.*/
```

```
//Criado por: Rodrigo Franklin Frogeri
```

```
#include <stdio.h> /*Esta linha indica ao pré-processador incluir no  
programa.
```

```
    O cabeçalho-padrão de entrada e saída*/
```

```
int main (void)
```



```
/*a palavra INT antes do MAIN indica que MAIN retorna um valor  
numérico inteiro. O void indica que MAIN não recebe nenhuma  
informação */
```

```
{
```

```
    printf ("Bacharelado em Sistemas de Informação – UNIS/MG \n");
```

```
    /* O \ (barra) encontrado em uma string indica ao compilador  
    que é caractere de escape, ou seja, que algo deve ser  
    realizado. No caso do \n em uma string colocará o cursor no  
    início da próxima linha.*/
```

```
    return 0;
```

```
}
```

ERRO COMUM EM PROGRAMAÇÃO 03:



Esquecer de colocar (&) antes de uma variável em uma instrução *scanf* quando isso deveria ocorrer.

Execute os programas abaixo e veja a diferença entre eles. Atente para os caracteres de escape utilizados nas strings.

```
/*Programa 03 - Apresenta duas strings na tela */
```

```
//Criado por: Rodrigo Franklin Frogeri
```

```
#include <stdio.h> /*Esta linha indica ao pré-processador incluir no  
programa
```

```
    O cabeçalho-padrão de entrada e saída*/
```

```
int main (void)
```

```
/*a palavra INT antes do MAIN indica que MAIN retoma um valor  
numérico inteiro. O void indica que MAIN não recebe nenhuma  
informação */
```

```
{
```

```
    printf ("Bacharelado em Sistemas de Informacao – UNIS/MG");
```



```
printf ("Disciplina de Algoritmo e Logica de Programacao.");
```

/* O \ (barra) encontrado em uma string indica ao compilador que é caractere de escape, ou seja, que algo deve ser realizado. No caso do \n em uma string colocará o cursor no início da próxima linha.*/

```
return 0;
```

```
}
```

DICA DE PROGRAMAÇÃO 02:



Recue em um nível o corpo inteiro de cada função (recomenda-se) dentro das chaves que definem o corpo da função. Esse recuo destaca a estrutura funcional dos programas e ajuda a torná-los mais fáceis de serem lidos.

/*Programa exemplo 04 - Apresenta duas strings na tela.

Criado por: Rodrigo Franklin Frogeri */



```
#include <stdio.h>
```

```
int main (void)
```

```

{
    printf ("Bacharelado em Sistemas de Informacao –
    UNIS/MG \n");

    printf ("Disciplina de Algoritmo e Logica de
    Programacao." \n");

    /* O \ (barra) encontrado em uma string indica ao
    compilador que é caractere de escape, ou seja, que algo
    deve ser realizado. No caso do \n em uma string colocará o
    cursor no início da próxima linha.*/

    return 0;
}

```

ERRO COMUM EM PROGRAMAÇÃO 04:



Sempre declare todas as variáveis que for usar no programa logo no início, a declaração de variáveis entre as instruções de execução causa erro no programa escrito em C.

Observação: Algumas linguagens aceitam declaração entre as instruções de execução.

Vamos analisar agora um exemplo mais elaborado.



```
#include <stdio.h>
main ()
{
    int Dias;
    float Anos;
    printf ("Informe a quantidade de dias: ");
    scanf ("%d", &Dias);
    Anos = Dias /365.25    /*Realizando a conversão de dias para anos*/
    printf ("\n \n%d dias equivalem a %f anos.\n", Dias, Anos);
}
```

Explicando o programa anterior:



Inicialmente são declaradas duas variáveis “Dias e Anos”, em que a primeira é um inteiro e a segunda aceita números com casas decimais “float”. A função “printf” solicita que o usuário informe uma quantidade de dias. Logo após os dados imputados pelo usuário são atribuídos a variável “Dias”, por meio do comando “scanf”. O “%d” dentro do scanf indica que será lido um valor inteiro. O segundo parâmetro “&Dias”, indica que o valor digitado pelo usuário será alocado a esta variável.



Atenção: Sempre que fizer uso do `scanf`, utilize o `&` antes da variável. Em sequência, é feito um cálculo simples. Por fim temos a função `printf` com três argumentos: `\n\n` corresponde a duas quebras de linha e o `%d` que será colocado um inteiro na tela mais a string `" dias equivalem a "` `%f` que significa que será colocado valor com ponto flutuante, ou seja, com casas decimais. No final há mais uma quebra de linha.

DICA DE PROGRAMAÇÃO 03:



Toda função deveria começar com um comentário e a descrição da finalidade da função.

Vamos analisar um outro programa em C que soma dois números inteiros.

```
/*Programa 05 - Soma dois números inteiros*/
```

```
#include <stdio.h>
```



```
int main(void)
```

```
{
```

```
int inteiro1;
```

```
int inteiro2;
```

```
int soma;

printf ("Digite o primeiro numero:\n");
scanf ( "%d", &inteiro1);

printf ("Digite o segundo numero:\n");
scanf ("%d", &inteiro2);

soma = inteiro1 + inteiro2;

printf ("A soma e: %d\n", soma);
return 0;
}
```

ERRO COMUM EM PROGRAMAÇÃO 04:



Confundir o operador de igualdade (==) com o de atribuição (=).

Bom pessoal, com este último exemplo encerramos essa breve introdução sobre a linguagem C. O que podemos perceber é que se você tem uma base sólida no desenvolvimento de algoritmos, independêr a linguagem de programação que você for utilizar, os passos serão os mesmos.

Compreender uma linguagem de programação a fundo, em todos os seus detalhes, leva um certo tempo e só é viável a partir do momento em que você tem necessidade daquele recurso. Por exemplo, a linguagem C dá suporte para interfaces gráficas, porém raramente é usada, mas caso seja preciso, você terá disponível.

O que quero dizer é que você precisa conhecer os principais conceitos da linguagem para desenvolver programas utilizando-a. . Sintaxe, estruturas de controle condicional e de repetição são princípios essenciais para qualquer linguagem de programação. Somente com essas informações a maioria dos problemas poderão ser resolvidos.

Finalizo o nosso guia de Algoritmos e Lógica de Programação com mais duas dicas de boas práticas de programação.

BOA PRÁTICA DE PROGRAMAÇÃO 02:



Nomes de variável com várias palavras podem ajudar a tornar um programa mais legível. Evite juntar palavras como em `relatoriofinal`. Em vez disso, separe as palavras com underline, como em `relatorios_finais`.

BOA PRÁTICA DE PROGRAMAÇÃO 03:



A primeira letra de um identificador deve ser sempre minúscula.
Exemplo: `sub_Total`, `soma_Produtos`.

Referência Bibliográfica



BUFFONI, S. **Apostila de Algoritmo Estruturado**. 4. ed. São Paulo: FIAA, 2003.

CORMEN, T. H. et al. **Algoritmos: Teoria e prática**. Rio de Janeiro: CAMPUS, 2012.

DEITEL, P.; DEITEL, H. **C como programar**. São Paulo: Pearson Prentice Hall, 2011.

GUEDES, S. **Lógica de Programação Algorítmica**. São Paulo: PEARSON, 2014.

MEDINA, M.; FERTIG, C. **Algoritmos e Programação – Teoria e Prática**. São Paulo: Novatec, 2005.